



Tessent BoundaryScan User's Manual

For Use with Tessent® Shell

Software Version 2015.4

December 2015

© 2015 Mentor Graphics Corporation
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

U.S. GOVERNMENT LICENSE RIGHTS: The software and documentation were developed entirely at private expense and are commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Accordingly, pursuant to FAR 48 CFR 12.212 and DFARS 48 CFR 227.7202, use, duplication and disclosure by or for the U.S. Government or a U.S. Government subcontractor is subject solely to the terms and conditions set forth in the license agreement provided with the software, except for provisions which are contrary to applicable mandatory federal laws.

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the owner of the Mark, as applicable. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/trademarks.

The registered trademark Linux[®] is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: supportnet.mentor.com/

Send Feedback on Documentation: supportnet.mentor.com/doc_feedback_form

Table of Contents

Chapter 1

Introduction	7
Benefits of Boundary Scan	7
Boundary Scan Overview	7
Simple Boundary Scan Architecture	8
Embedded Boundary Scan	11
TAP Controller State Machine	12
Boundary Scan Insertion With Tessent BoundaryScan	13

Chapter 2

Getting Started	15
DFT Flow Using Tessent Shell	16
Prerequisites	16
Design Flow Dofile Example	17
Design Loading	18
Set the Context	18
Read the Libraries	19
Read the Design	20
Elaborate the Design	20
Specify and Verify DFT Requirements	20
Specify DFT Specification Requirements	21
Add Constraints	22
Run DRC	24
Create DFT Specification	24
Invoke create_dft_specification	24
Edit/Configure the DFT Specification According to Your Requirements	25
Validate the DFT Specification	32
Process DFT Specification	32
Create DFT Hardware with the DFT Specification	33
Extract ICL	33
Preparation for Pattern Generation	34
Create Patterns Specification	34
Automatically Created Patterns Specification	35
Edit / Configure the Patterns Specification According to Your Requirements	35
Process Patterns Specification	37
Create Patterns / Test Benches According to Your Specification	37
Run and Check Test Bench Simulations	38
Run Simulations	38
Check Results	39
Test Logic Synthesis	39
RTL Design Flow Synthesis	39
Extract SDC	39

Synthesizing the RTL Design with Test Logic	40
Gate Level Design Flow Synthesis	41
Run Synthesis	41
Concatenate Netlist Generation	41
Chapter 3	
Boundary Scan Specific Topics	43
Pad Cell Library	43
Using <i>pad.library</i> and Verilog Simulation Models for the Pad Cells.	43
Creating a Tessent Cell Library from <i>pad.library</i> and Verilog Simulation Models.	44
Inserting Tessent Boundary Scan on a Custom or Pre-Existing TAP Controller	45
Sharing TAP Ports Between a Pre-Existing TAP and Tessent TAP	46
AC JTAG	48
Embedded Boundary Scan (EBscan)	48
Adding a Test Data Register (TDR) to the TAP Controller	49
BSDL-Only Flow	52
Dividing Boundary Scan for Logic Test	54
Multiple Bonding Configurations	56
Custom Boundary Scan Cells	59
Debugging Failing JtagBscan Simulations	61
Chapter 4	
MemoryBIST Insertion with BoundaryScan	65
Overview	65
TAP, BoundaryScan and MemoryBIST	66
MemoryBIST Insertion Before Tap and BSCAN	68
Chapter 5	
Tap, BoundaryScan and LPCT Type 2 TestKompress	71
Overview	71
Design Flow	72
TAP and Boundary Scan Insertion	73
Scan Chain Insertion and Stitching	76
EDT (Type 2 LPCT) IP Creation	79
Pattern Generation and Simulation	80
Appendix A	
Tessent Core Description	83
Core	85
BoundaryScan	86
Interface	87
ExternalPort	90
Cell	95

Table of Contents

Third-Party Information

End-User License Agreement

List of Figures

Figure 1-1. Boundary Scan Chips on Board	8
Figure 1-2. Boundary Scan Architecture	9
Figure 1-3. Embedded Boundary Scan Implementation	12
Figure 1-4. TAP Controller Finite State Machine Diagram	13
Figure 2-1. Design Flow for Tessent Shell Boundary-Scan	16
Figure 2-2. Design Loading	18
Figure 2-3. Specify and Verify DFT Requirements	21
Figure 2-4. Create DFT Specification	24
Figure 2-5. GUI to Edit DFT Specification	27
Figure 2-6. Process DFT Specification	32
Figure 2-7. Extract ICL	34
Figure 2-8. Create Patterns Specification	35
Figure 2-9. Process Patterns Specification	37
Figure 2-10. Run and Check Test Bench Simulations	38
Figure 2-11. Test Logic Synthesis	39
Figure 3-1. Adding a TAP with DFTVisualizer	51
Figure 3-2. Viewing the Added TDR in DFTVisualizer	52
Figure 3-3. Example PatternsSpecification for Third Party BSDL	53
Figure 3-4. Tessent Shell dofile for Third Party BSDL Processing	53
Figure 3-5. Adding Bonding Configurations in DFTVisualizer	57
Figure 3-6. Configuring Bonding Options in DFTVisualizer	58
Figure 3-7. Simulation Output Directory Contents	62
Figure 4-1. Design Flow for Tessent Shell	66
Figure 5-1. Tap, Boundary Scan and LPCT Type 2 TK Design Flow	71
Figure 5-2. Tap, Boundary Scan and Type 2 LPCT TestKompress Implementation	72
Figure 5-3. post_dft_insertion_procedure.tcl Example File (design name = cpu_top)	75
Figure 5-4. Example cpu_top_gate_tessent_tap_main.pdl File	77
Figure 5-5. Example cpu_top_gate_tessent_tdr_logic_enable.pdl File	77
Figure 5-6. Example e_chains.dofile	78
Figure 5-7. Example existing_chains.testproc File	78
Figure A-1. Bidirectional Pad with Data and Enable Cell	93

Chapter 1

Introduction

Boundary scan, sometimes referred to as JTAG (for Joint Test Action Group, the committee that formulated the IEEE standard 1149.1 that describes boundary scan), is a Design for Test (DFT) technique that facilitates the testing of printed circuit board (and Multi Chip Module or MCM) interconnect circuitry and, to a limited extent, the chips on those boards. Boundary scan test structures greatly improve board-level testing, thus shortening the manufacturing test and diagnostics processes.

The most recent revision to the standard occurred in 2013 and is known as IEEE 1149.1-20013. Support for differential and capacitively coupled pin testing was introduced with the IEEE 1149.6 specification. Tessent BoundaryScan is fully compliant with the IEEE 1149.1-2001 and IEEE 1149.6 specifications, but does not support the extensions introduced in the IEEE 1149.1-2013 specification.

Benefits of Boundary Scan	7
Boundary Scan Overview	7
Simple Boundary Scan Architecture.	8
Embedded Boundary Scan	11
TAP Controller State Machine	12
Boundary Scan Insertion With Tessent BoundaryScan.	13

Benefits of Boundary Scan

As the usefulness of in-circuit test diminishes owing to the increasing popularity of surface mount devices, boundary scan provides the same benefits, but without requiring physical access to each electrical network. Adding boundary scan logic to your board lets you detect the vast majority of board manufacturing process faults using boundary scan test methods. These faults include wrong components, missing components, incorrectly oriented components, components with stuck pins, shorts, opens, and blown wire bonds.

Although your engineering costs may increase slightly because of the additional silicon and ports used for the boundary scan circuitry, implementing the IEEE 1149.1 standard can dramatically reduce a design's manufacturing costs.

Boundary Scan Overview

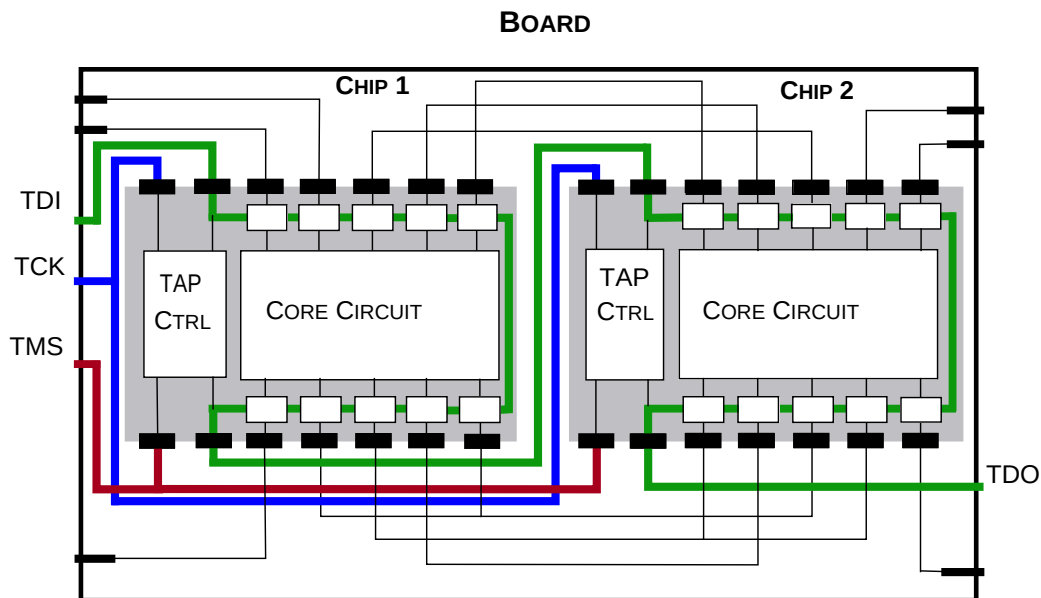
When used on a board, boundary scan provides access to the input and output ports of the chips by linking them together into a long scan path. Data shifts along the scan path, starting at the board's input TDI (test data in) and ending at the board's output TDO (test data out). In

between, the scan path connects all the devices on the board that contain boundary scan circuitry. The TDO of one chip feeds the TDI of the next, all the way around the board. The inputs, TCK (test clock) and TMS (test mode select), connect in parallel to each boundary scan device in the scan path of the board. With this configuration you can test board interconnections, perform a snapshot of normal system data, or test individual chips. The TAP (test access port) controller is a state machine that controls the operation of the boundary scan circuitry.

Boundary scan circuitry's primary use is in board-level testing, but it can also control circuit-level test structures, such as BIST or internal scan. By adding boundary scan circuitry to your design, you create a standard interface for accessing and testing chips at the board level.

Figure 1-1 shows a board containing two chips with boundary scan circuitry.

Figure 1-1. Boundary Scan Chips on Board



Simple Boundary Scan Architecture

This section discusses boundary scan circuitry in detail.

Figure 1-2. Boundary Scan Architecture

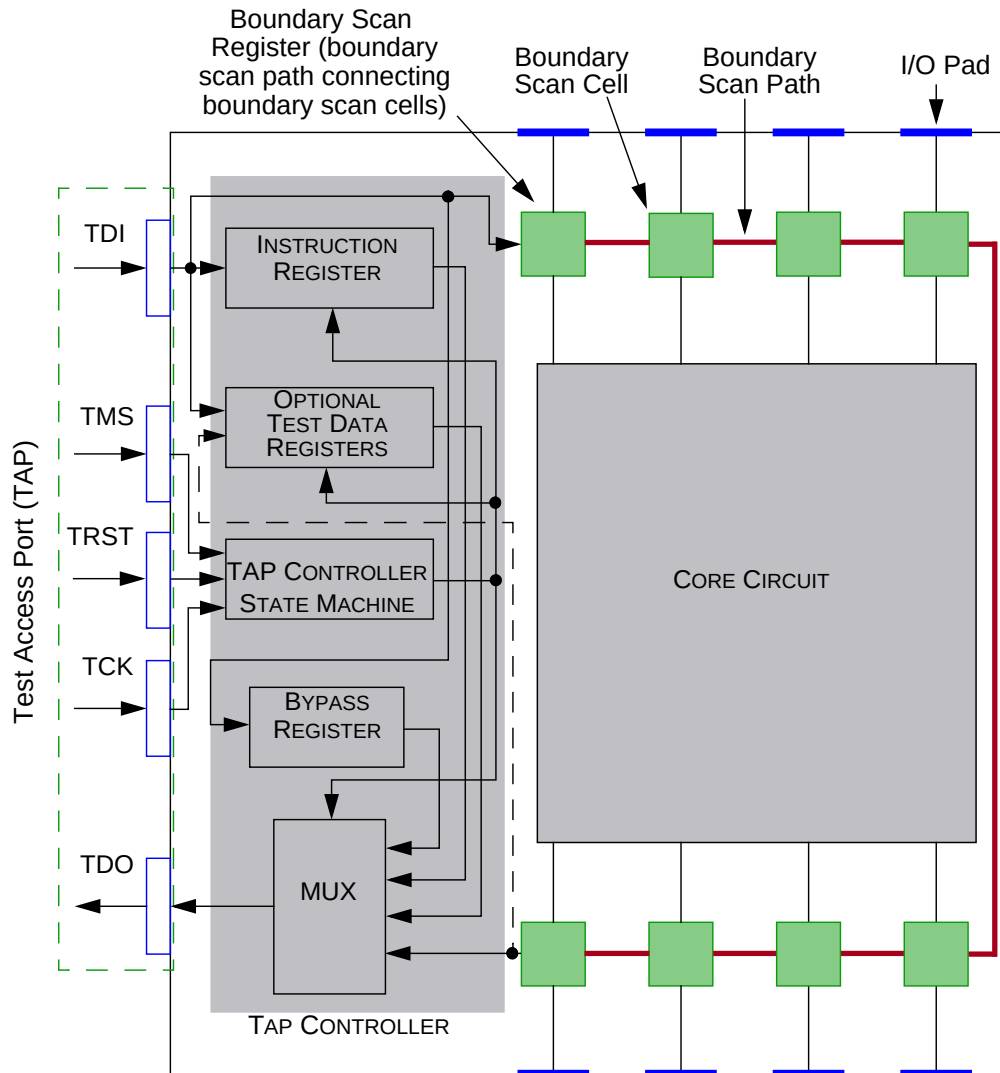


Figure 1-2 shows the general configuration of a chip after the addition of boundary scan logic. This simple boundary scan architecture contains the following components:

- **Core Circuit** — This is the application logic of the original design before adding boundary scan logic. This logic might already contain internal scan circuitry (or at least internal scan ports so boundary scan circuitry can connect to the internal scan circuitry).
- **Boundary Scan Cells** — These contain memory elements for capturing data from the circuit, loading data into the circuit, or serially shifting data to the next scan cell in the path. The tool places boundary scan cells between the core logic and each input, bidirectional, and 2- or 3-state output pin. Boundary scan cells collectively comprise a

parallel-in, parallel-out shift register that runs along the periphery, or boundary, of the original design.

- **Test Access Port (TAP)** — The TAP consists of a minimum of four pins for the four signals that make up the test bus. These signals include the test clock (TCK), the test data input (TDI), the test data output (TDO), and the test mode selector (TMS). Also shown is an optional, active low, asynchronous test reset (TRST).
- **TAP Controller State Machine** — A finite state machine that controls the operation of the instruction and test data registers. The TAP controller's state depends on the value of the TMS line at each clock pulse and the controller's current state.
- **Boundary Scan Register** — Considered the main test data register, the boundary scan register is a virtual shift register (consisting of the connection of the individual boundary scan cells) that can, either serially or in parallel, load and unload input and output data for the circuit.
- **Bypass Register** — The bypass register shortens the serial path between TDI and TDO to one cell when there is no requirement to test a particular device. This shortened path in effect bypasses the chip, allowing more efficient data shifting to other ICs in the chain.
- **Optional Test Data Registers:**
 - **Device Identification Register** — This register contains a device identification code or programming code used to check that the board has proper chips.
 - **Data-Specific Registers** — These registers allow access to the chip's test support features, such as BIST and internal scan paths.
- **Instruction Register** — The instruction register controls the boundary scan circuitry by connecting a specific test data register between the TDI and TDO pins. It controls the operation affecting the data in that register, using a predefined set of instructions. Some instructions are mandatory, and others are optional.

The mandatory instructions include:

- **EXTEST** — This instruction tests circuitry external to the ICs themselves, such as board interconnect. EXTEST is the main test instruction for boundary scan testing.
- **SAMPLE/PRELOAD** — This instruction takes data from the chips's I/O pads and latches it in the boundary scan register (during normal board operation). The IEEE Std. 1149.1-2001 has redefined this instruction as two separate instructions (SAMPLE and PRELOAD).
- **SAMPLE** — This instruction takes data from the chip's I/O pads and latches it in the boundary scan register (during normal board operation).
- **PRELOAD** — This instruction allows the loading of data into the boundary scan register before selecting another instruction.

- **BYPASS** — This instruction enables bypassing of chips not being tested. For example, if you only want to test one chip in a board with 20 chips in the boundary scan chain, all bypassed chips will contribute a single scan flop to the scan chain and the chip under test would contribute all boundary scan registers to the scan chain. Consequently, shifted data needs to go through only one extra shift to pass through each of the chips not being tested (as opposed to the entire boundary scan register).
- **EXTEST_PULSE** — This instruction tests circuitry external to the ICs themselves, for designs that have AC pins. This instruction is mandatory only when using AC cell types.

The optional instructions include:

- **INTTEST** — This instruction tests a chip's internal circuitry by applying a test vector to, and capturing the output response from, the application logic.
- **IDCODE** — This instruction connects the device identification register between TDI and TDO. The device identification register contains the device ID code, which is normally used to determine if the chip belongs on the board.
- **USERCODE** — This instruction also selects the identification register, but the information placed in that register is now user-defined and is meant to expand on the IDCODE information.
- **CLAMP** — This instruction forces static 1s or 0s on selected nodes in order to create a testable situation or block interfering signals.
- **HIGHZ** — This instruction forces an IC's output and bidirectional pins into a high-impedance state. In this condition, an in-circuit tester can test it safely without the potential for overdrive damage.
- **RUNBIST** — This instruction executed the circuit's internal BIST procedure.
- **EXTEST_TRAIN** — This instruction, which is available when using AC cell types, operates similarly to EXTEST_PULSE but can use multiple TCK cycles. For more detail, refer to IEEE 1149.6, available through the IEEE.

Embedded Boundary Scan

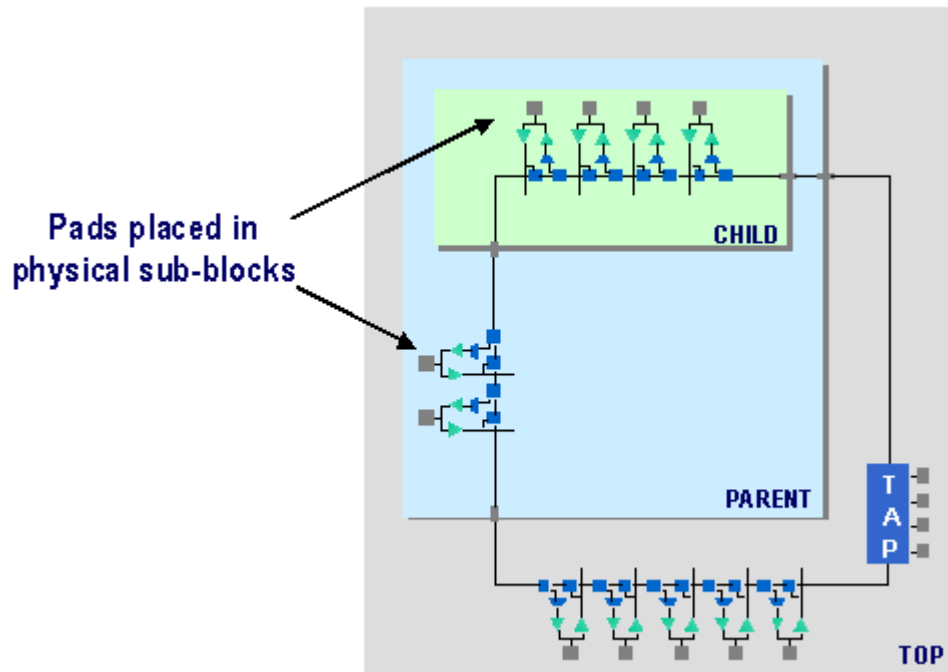
Increasingly, chip I/O cells are being placed directly into cores in order to be closer to the logic they service. This approach typically results in significant physical design benefits including reduced signal routing and improved timing. Mentor Graphics' embedded boundary scan feature allows boundary scan cells to be integrated alongside their associated I/O cells within the core rather than at the top level of the chip.

Boundary scan cells can be added to any core at any level within a design. A three -level example is shown in [Figure 1-3](#).

The embedded boundary scan capability of Tessent BoundaryScan automates both the integration of the boundary scan cells and verification of the resulting boundary scan segment within the core.

This approach not only maintains and extends the physical design benefits of placing I/O cells directly into cores, but also enables a more efficient core re-use methodology as all design and DFT sign-off activity can take place fully at the core level.

Figure 1-3. Embedded Boundary Scan Implementation

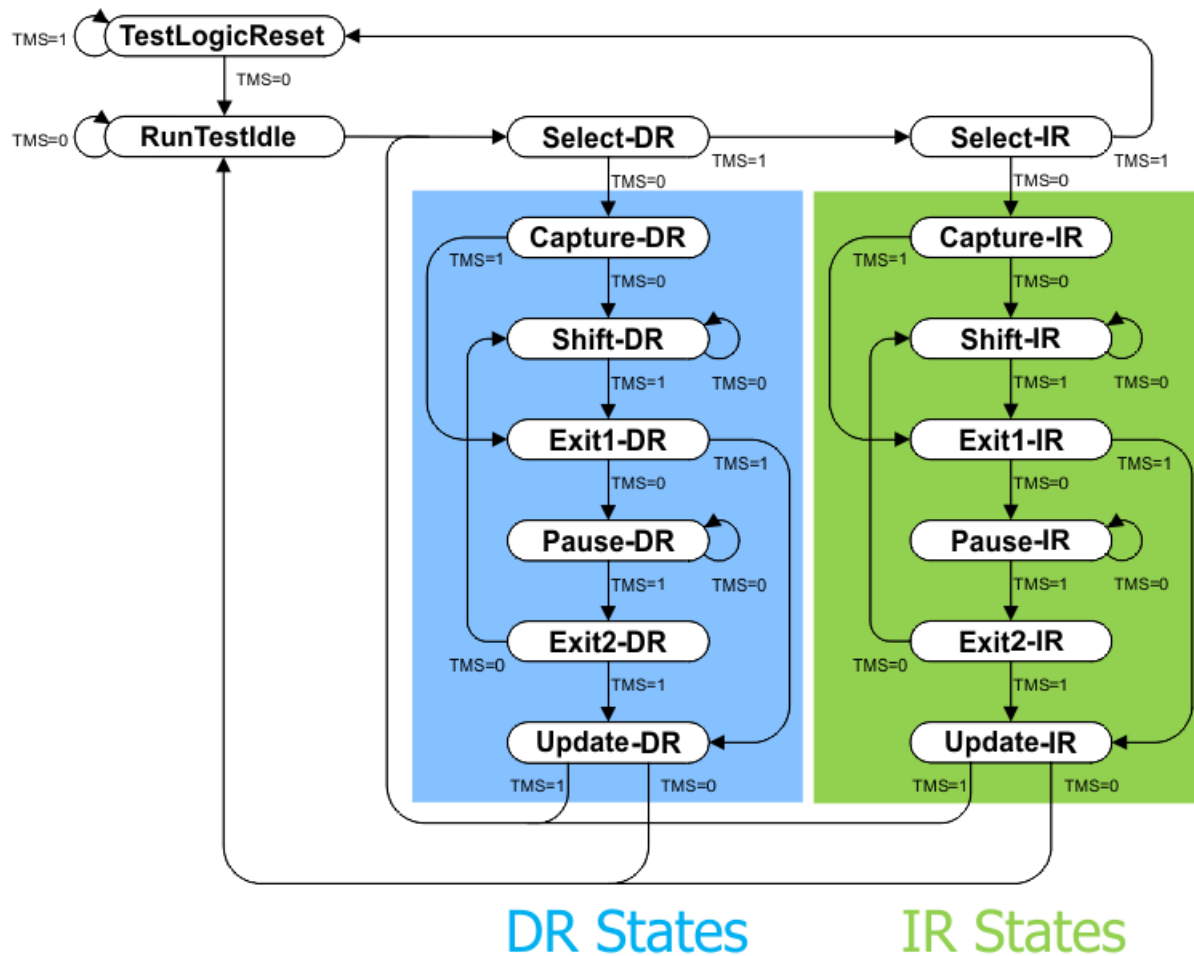


TAP Controller State Machine

The TAP controller is a synchronous finite state machine that controls the operation of the instruction and test data registers. The TAP controller's state depends on the value of the TMS line at the rising edge of each TCK clock pulse and the controller's current state.

Figure 1-4 shows the finite state machine diagram for the TAP controller of a IEEE 1149.1 circuit.

Figure 1-4. TAP Controller Finite State Machine Diagram



The TMS signal (the value shown adjacent to each state transition) controls the state transitions on each rising edge of TCK. The rising edge of the TCK clock also captures the TAP controller inputs.

Boundary Scan Insertion With Tessent BoundaryScan

Tessent BoundaryScan is the Mentor Graphics boundary scan insertion tool. The tool creates and interconnects RTL-level boundary scan logic compliant with the IEEE 1149.1-2001 standard.

Features of the Tessent BoundaryScan tool:

- **Instruction support** — Full support of required IEEE standards 1149.1 instructions.

- **Extension support** — Support of base extensions to IEEE 1149.1, such as the Device ID register.
- **Compliant Verilog** — Generation of Verilog (IEEE 1364-2001) that is compliant with the ModelSim (Verilog), Synopsys' Design Compiler, and other industry synthesis tools.
- **RTL-level boundary scan generation** — Insertion and interconnection of boundary scan circuitry at the RTL level, moving generation of test circuitry to earlier in the design process.
- **Customized boundary scan** — Generation of default or user-customized boundary scan architectures.
- **I/O pad synthesis** — Generation of generic I/O pads or technology-specific I/O pads.
- **Automatic connection** — Automatic connection of boundary scan to internal scan logic.
- **Test bench generation** — Generation of a test bench, which allows testing of the boundary scan logic after interconnection with the core application logic.
- **Test vector generation** — Generation of boundary scan test vectors in a variety of ASIC vendor test data formats, as well as ASCII, binary, STIL, WGL and other pattern formats.
- **Setup file generation** — Generation of ATPG setup files, for designs with generated boundary scan circuitry controlling internal scan circuitry.
- **Compliant BSDL** — Production of BSDL output that is compliant with IEEE standard IEEE 1149.1-2001 specification.
- **Generic element mapping** — Mapping of boundary scan elements to generic boundary scan library cells (which enhances re-targetability of the boundary scan circuitry).
- **Technology-specific element mapping** — Mapping of boundary scan, I/O pad, and TAP controller elements to technology-specific library cells. Technology mapping provides support for replacing generic boundary scan cells, I/O pads, and the TAP controller by equivalent technology-specific cells.

Chapter 2

Getting Started

This chapter describes how to start inserting Tessent BoundaryScan within Tessent Shell and includes examples showing the most common scenarios and usages.

For a complete set of wrapper and property descriptions, refer to the “BoundaryScan” section of the “[DftSpecification Configuration Syntax](#),” “[PatternsSpecification Configuration Syntax](#),” and “[DefaultsSpecification Configuration Syntax](#)” sections in the *Tessent Shell Reference Manual*. The flow and steps are the same for inserting TAP, boundary bscan, or embedded boundary scan. Each step describes any commands that are required for embedded boundary scan.

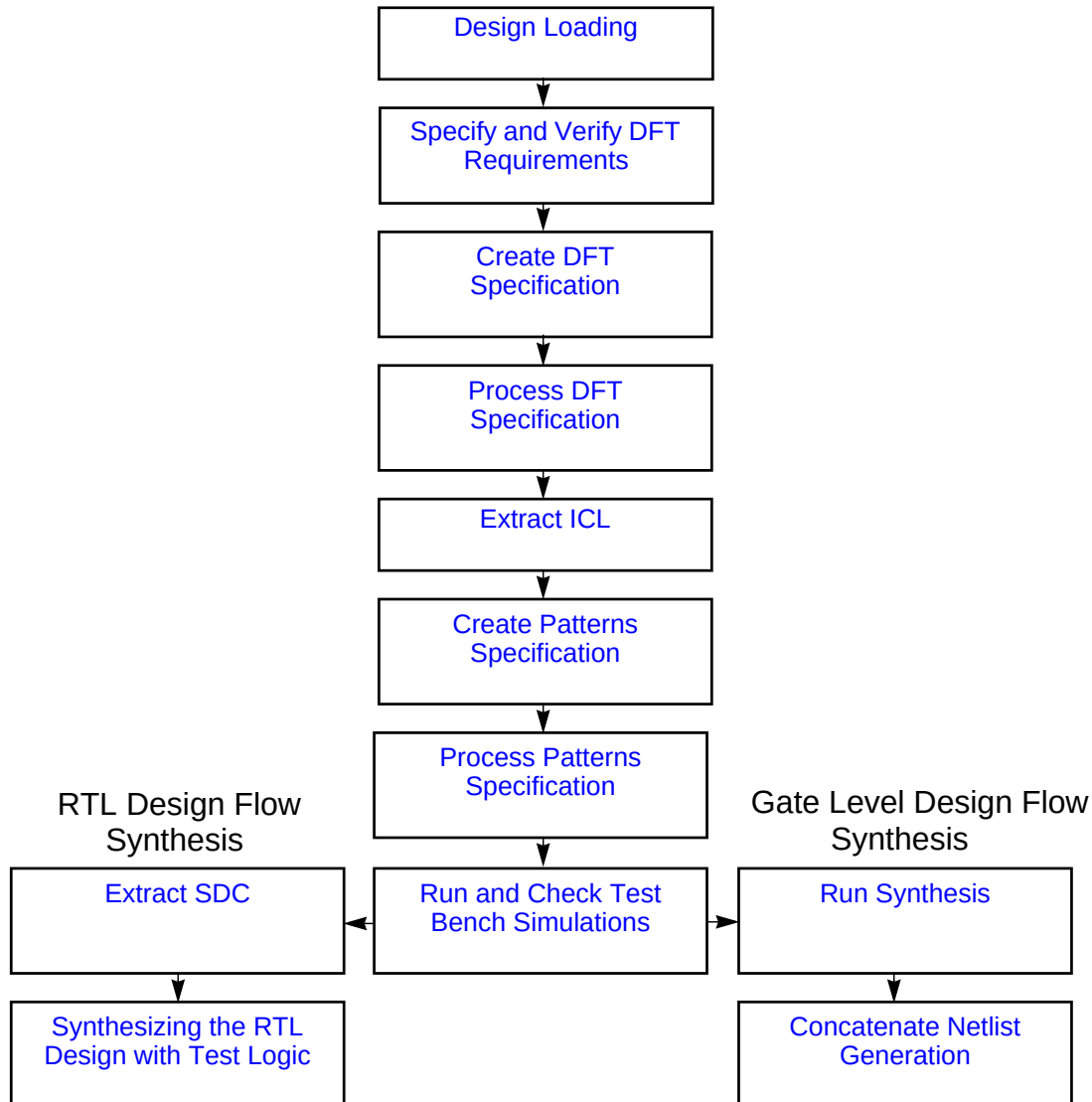
DFT Flow Using Tessent Shell	16
Design Loading.....	18
Specify and Verify DFT Requirements	20
Create DFT Specification	24
Process DFT Specification.....	32
Extract ICL.....	33
Create Patterns Specification	34
Process Patterns Specification	37
Run and Check Test Bench Simulations	38
Test Logic Synthesis.....	39

DFT Flow Using Tessent Shell

Tessent BoundaryScan in Tessent Shell has a basic, high-level flow sequence.

[Figure 2-1](#) illustrates the high-level sequence of steps required to insert Tessent BoundaryScan into a design. Each step in the figure links to more detailed information about the design-for-test flow, including examples.

Figure 2-1. Design Flow for Tessent Shell Boundary-Scan



Prerequisites

To use this flow, you must have either an RTL or a gate-level netlist with IO pads already inserted into the design. For an RTL netlist, you must have the Tessent cell library or the pad

library for the pad cells. For a gate-level netlist, you must have the Tessent cell library or the ATPG library for the standard cells, in addition to the Tessent cell library for the IO pad cells. The pad library supported by Tessent BoundaryScan-LV is natively supported in Tessent Shell if the Tessent cell library for the IO pad cells is not present.

Design Flow Dofile Example

The example dofile in this section shows you how to set up a typical design flow. Subsequent sections of this chapter will explain each step of the design flow in more detail.

The following example dofile follows the design flow described in [Figure 2-1](#).

Design Loading

```
set_context dft -rtl
read_cell_library ../library/adk_complete.tcelllib
read_verilog ../netlist/cpu_top.v
set_current_design cpu_top
```

Specify and Verify DFT Requirements

```
set_dft_specification_requirements -boundary_scan on
set_design_level chip
set_attribute_value tck_p -name function -value tck
set_attribute_value tdi_p -name function -value tdi
set_attribute_value tms_p -name function -value tms
set_attribute_value trst_p -name function -value trst
set_attribute_value tdo_p -name function -value tdo
set_boundary_scan_port_options ramclk_p -cell_options clock
set_boundary_scan_port_options reset_p -cell_options sample
check_design_rules
```

Create DFT Specification

```
set spec [create_dft_specification]
report_config_data $spec
```

Process DFT Specification

```
process_dft_specification
```

Extract ICL

```
extract_icl
```

Create Patterns Specification

```
create_patterns_specification
```

Process Patterns Specification

```
process_patterns_specification
```

Run and Check Testbench Simulations

```
set_simulation_library_sources -y ../library/verilog \  
-v ../library/pad_cells.v  
run_testbench_simulations  
check_testbench_simulations -report_status
```

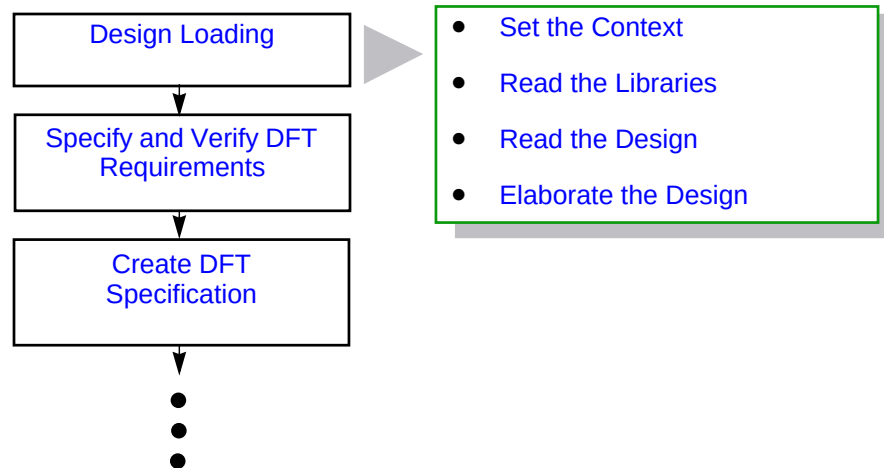
Test Logic Synthesis

```
extract_sdc  
run_synthesis
```

Design Loading

Design Loading is the first step in Tessent BoundaryScan insertion using Tessent Shell. The step consists of setting the correct context, reading libraries, reading the design, and elaborating the design.

Figure 2-2. Design Loading



Set the Context

In Tessent Shell, setting the context means two things. First and foremost, you must set the context to dft for boundary scan hardware to be created. Second, you must specify whether the design type to be read in is written in RTL. If so, you must specify the `-rtl` option. If the design to be read in is a gate-level Verilog netlist, you should specify the `-no_rtl` option. When using the `-no_rtl` mode, a concatenated netlist is written out at the end of the dft insertion phase. In rtl mode, the file structure of the input design is preserved and only the modified design files are written out at the end of the dft insertion phase along with the newly created test IP. The netlist to be read in can be Verilog, VHDL, or mixed language.

If Tessent tools inserted [embedded boundary scan](#), which is now being integrated at the next level, you must open the Tessent Shell Data Base (TSDB) of the child (the embedded boundary

scan's sub_block or physical_block) using the [open_tsdb](#) command. If you are using the same TSDB for both child and parent, you can reuse the TSDB (the default is tsdb_outdir), and you do not need to explicitly open the default TSDB because the existing content of the TSDB output directory is automatically visible to the tool. See the [set_tsdb_output_directory](#) command description for how to control the name and location of the TSDB output directory.

Example 1

The following example sets the context to dft and specifies that the design to be read in is written in RTL.

```
set_context dft -rtl
```

Example 2

The following example sets the context to dft and specifies that a gate-level netlist will be read in.

```
set_context dft -no_rtl
```

Example 3

The following example opens a child's TSDB directory and therefore, exposes it at the parent level.

```
open_tsdb ../ebscan_tsdb_outdir
```

Read the Libraries

You can use the read_cell_library command to read in the library file for the pad IO macros that are instantiated in the design. If you are inserting Tessent BoundaryScan into an RTL netlist or design, reading the library for the pad IO macros is sufficient. If the Tessent cell libraries do not include the pad information, the legacy LV pad library format is natively supported by the read_cell_library command and can be used to augment the Tessent cell libraries with the pad information.

Example 1

The following example reads in the Tessent cell library file for the pad IO macros.

```
read_cell_library ../library/adk_complete.tcelllib
```

Example 2

The following example reads in the ATPG.lib files and the old pad library description when the Tessent cell libraries do not include the pad information.

```
read_cell_library ../library/atpg.lib
```

```
read_cell_library ../library/pad.library
```

Read the Design

In Tessent Shell, after setting the context and loading the required libraries, you can use the `read_verilog` command to read in the design.

Example 1

The following example reads in one netlist, which can be either RTL or gate level.

```
read_verilog ../netlist/cpu_top.v
```

Example 2

The following example reads in a verilog file and directory of design files.

```
set_design_sources -format verilog -V ../design/top.v \  
-Y ../design -extensions {v gv}
```

Elaborate the Design

The next step in Design Loading is to elaborate the design using the `set_current_design` command. The `set_current_design` command specifies the root of the design. If any module descriptions are missing, design elaboration will identify them. For Tessent BoundaryScan insertion, modules such as memory instances, PLL instantiations, etc. are not needed. You can specify them with the `add_black_box -module` command.

Example

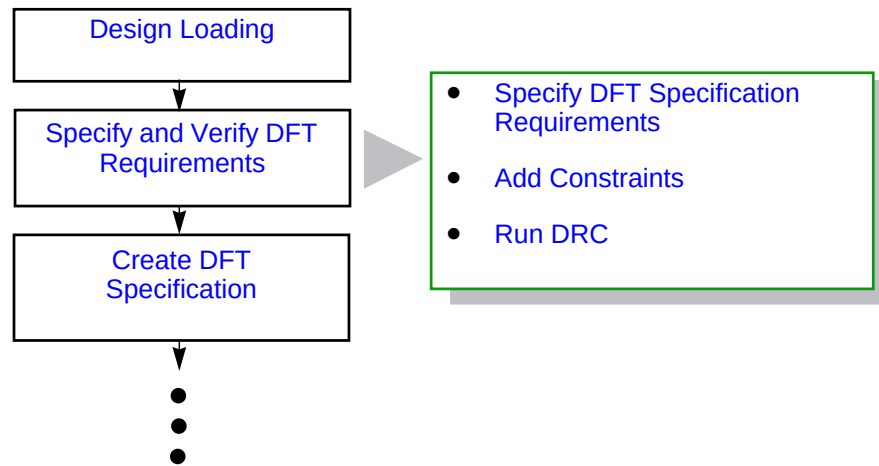
The following example shows how to use the `set_current_design` command.

```
set_current_design cpu_top
```

Specify and Verify DFT Requirements

The next step to insert Tessent BoundaryScan in Tessent Shell is to specify the DFT requirements, add constraints, and verify whether the DFT requirements specified are correct by running DRC (Design Rule Checking).

Figure 2-3. Specify and Verify DFT Requirements



Specify DFT Specification Requirements

To insert BoundaryScan, you must specify the DFT specification requirements with the `set_dft_specification_requirements` command. This provides a template for logic and hardware that will be added.

You specify *chip* with the `set_design_level` command when you are inserting boundary scan at the chip level. If you are using embedded boundary scan, you must specify either `sub_block` or `physical_region`. Also, you must use the `set_boundary_scan_port_options` command to specify the list of pad IO ports that need boundary scan. See [Example 2](#).

Example 1

The following example shows how the boundary scan DFT specification requirements are specified and how the design is specified at the chip level.

```
set_dft_specification_requirements -boundary_scan On
set_design_level chip
```

Example 2

The following example shows, for embedded boundary scan, how to provide a Tcl list of pad IO ports that need boundary scan cells to be inserted. The example also shows how to set the design level to sub-block.

```
set_dft_specification_requirements -boundary_scan on

set_boundary_scan_port_options -pad_io_ports [list in1 in_diff_p in_
diff_n out1 out_diff_p out_diff_n clk A Y]

set_design_level sub_block
```

Note



If using embedded boundary scan, you may need to use the [add_input_constraints](#) command to constrain some ports to either a 1 or 0 value so that the pad IO macros function properly. Typically, these are driven to the proper values at the next higher level in the design.

Add Constraints

To insert Tessent BoundaryScan, four TAP pins (TDI, TCK, TMS, and TDO) must be available at the chip level and connected to pad IO macros. TRST, which is optional, can be an output pin of a power-up detector.

When all five TAP pins are connected to chip level pad IO macros, a 5-pin TAP will be inserted. If TRST is connected to an internal power-on reset pin, a 4-pin TAP will be inserted.

Note



If you need to specify an internal pin for TRST, and that pin connects to a chip level pad IO macro, two insertion passes are needed. The TAP needs to be inserted in the first pass and the boundary scan insertion is performed in a subsequent pass.

The TAP pins can be identified in the constraints section of your dofile or can be specified in the pin order file. Similarly, the power and ground pins can be identified in the constraints section or specified in the pin order file. If any special boundary-scan cell types are required, they can also be specified in the constraints section using the `set_boundary_scan_port_options` command.

If you are using embedded boundary scan and some ports must be constrained to a constant 1 or 0 value, use the `add_input_constraints` command. Typically, these values are properly driven at the next higher level in the design.

Example 1

The following example specifies the function and purpose of the five TAP pins (tck_p, tdi_p, tms_p, trst_p, tdo_p).

```
set_attribute_value tck_p -name function -value tck
set_attribute_value tdi_p -name function -value tdi
set_attribute_value tms_p -name function -value tms
set_attribute_value trst_p -name function -value trst
set_attribute_value tdo_p -name function -value tdo
```

The following specifies the power and ground pins.

```
set_attribute_value vdd* -name function -value power
set_attribute_value vss* -name function -value ground
```

The following specifies special boundary scan cell types.

```
set_boundary_scan_port_options ramclk_p -cell_options clock
set_boundary_scan_port_options reset_p -cell_options dont_touch
```

Example 2

The following example shows how to use the [DefaultsSpecification](#) wrapper to specify the five TAP pins if they are the standard port names used across all designs. The DefaultsSpecification wrapper is used when the create_dft_specification command is issued in the next step.

```
DefaultsSpecification(group) { //Legal: company | group | user
  DftSpecification {
    IjtagNetwork {
      HostScanInterface {
        Chip {
          tck : tck_p;
          tdi : tdi_p;
          tdo : tdo_p;
          tms : tms_p;
          trst : trst_p;
        }
      }
    }
  }
}
```

You also can use the [set_defaults_value](#) command to set the DefaultsSpecification and the [get_defaults_value](#) command to see the specified value.

Example 3

The following example reads in the pin order file where the five TAP pins, power, and ground are specified.

```
set_boundary_scan_port_options -pin_order_file cpu_top.pinorder.my
```

Example 4

The following example shows how you can set some ports to a constant value if you are using embedded boundary scan.

```
add_input_constraints drive_strength[1] -C0
add_input_constraints drive_strength[2] -C0
add_input_constraints edriver1 -C1
add_input_constraints edriver2 -C1
```

Run DRC

The next step in Specify and Verify DFT Requirements is to run Design Rule Checking (DRC) to make sure all the constraints are correct. Once DRC is clean, Tessent Shell moves from the SETUP to the ANALYSIS prompt.

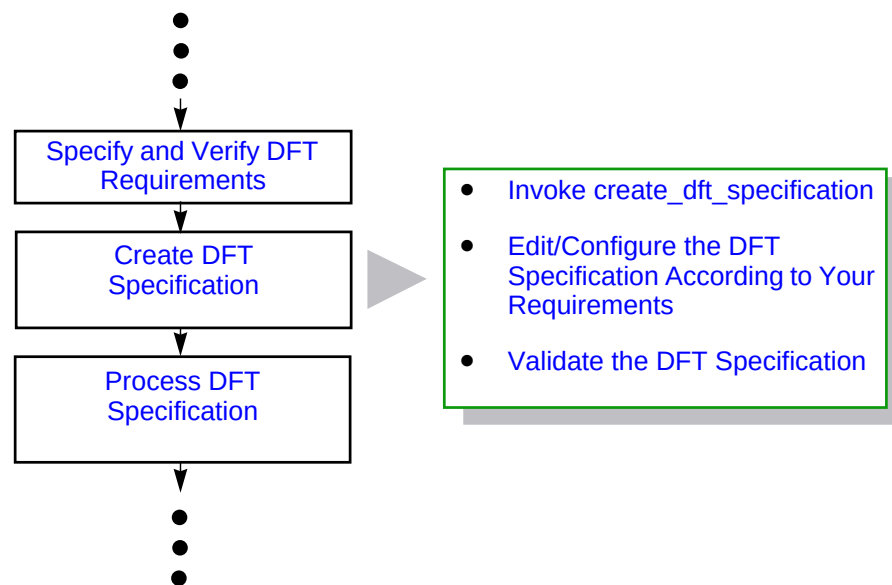
```
check_design_rules
```

Create DFT Specification

The next step is to create a DFT specification.

Use the [create_dft_specification](#) command to create a default DFT specification based on the DFT requirements specified in the previous step. You can use the [report_config_data](#) command to report this default DFT specification. Several different ways are available to edit or configure the DFT specification to meet the custom requirements.

Figure 2-4. Create DFT Specification



Invoke `create_dft_specification`

Based on the DFT specification requirements described in the previous step, a DFT specification is automatically created using the `create_dft_specification` command. This DFT specification is stored in memory.

To report the DFT specification in memory, use the [report_config_data](#) command. The DFT specification uses IJTAG network infrastructure because this is the only supported method for incremental insertion passes. The IJTAG network is fully compliant with the 1149.1 IEEE

standard. For further information about the Tessent IJTAG flow, refer to the [Tessent IJTAG User's Manual](#).

To insert boundary scan into a pre-existing TAP in your design, you must have an ICL for the TAP. The tool automatically uses this TAP to connect to the boundary scan chain if the TAP has ScanInterface host_bscan in the TAP ICL. You can, however, also specify which host_bscan to connect to by using the `create_dft_specification -existing_host_bscan_scan_in` command. The ICL for the pre-existing TAP is read in automatically if it is in a location where the module description of the TAP is present. You also can use the `read_icl` command to read the ICL for a pre-existing TAP. For requirements when using a pre-existing TAP with boundary scan, refer to the “Requirements on a TAP to be usable for BoundaryScan” section in the *Tessent Shell Reference Manual*.

Example 1

In the following example, the DFT specification generated with the `create_dft_specification` is stored in a variable called `dft_spec` so that the variable can be used to report the DFT specification.

```
set dft_spec [create_dft_specification]
report_config_data $dft_spec
```

Example 2

The following example shows how to connect to the pre-existing TAP when multiple TAPs are present. This only works if a ScanInterface host_bscan is in the ICL file for the pre-existing TAP controller. If a single TAP controller is present, the tool automatically uses this controller, and you do not need to provide the host_bscan to which to connect.

```
create_dft_specification -existing_host_bscan_scan_in \
    My_TAP_INST/fromBscan
report_config_data
```

`My_TAP_INST` in the above example is the instance name of the pre-existing TAP in the design, and `fromBscan` is the input port on the pre-existing TAP where the boundary-scan chain needs to connect.

Edit/Configure the DFT Specification According to Your Requirements

Use one of the following methods to edit or configure the created DFT specification according to your requirements. You do not need to edit the DFT specification if you want to use the default configuration the way it is.

Method 1: Use the GUI to edit the DFT specification

In this method, you use the GUI to edit the DFT specification that has been created. Open the GUI with the [display_specification](#) command. The GUI displays the DFT specification based on the DFT requirements specified in the [Specify and Verify DFT Requirements](#) step. The edits you make and apply with the GUI update the DFT specification in memory. If you want the flow to work next time without the GUI edits and be more script-based, you can use the `report_config_data` command to display the edits you made, and then you can add them to the DFT specification using the [read_config_data](#) command.

Example

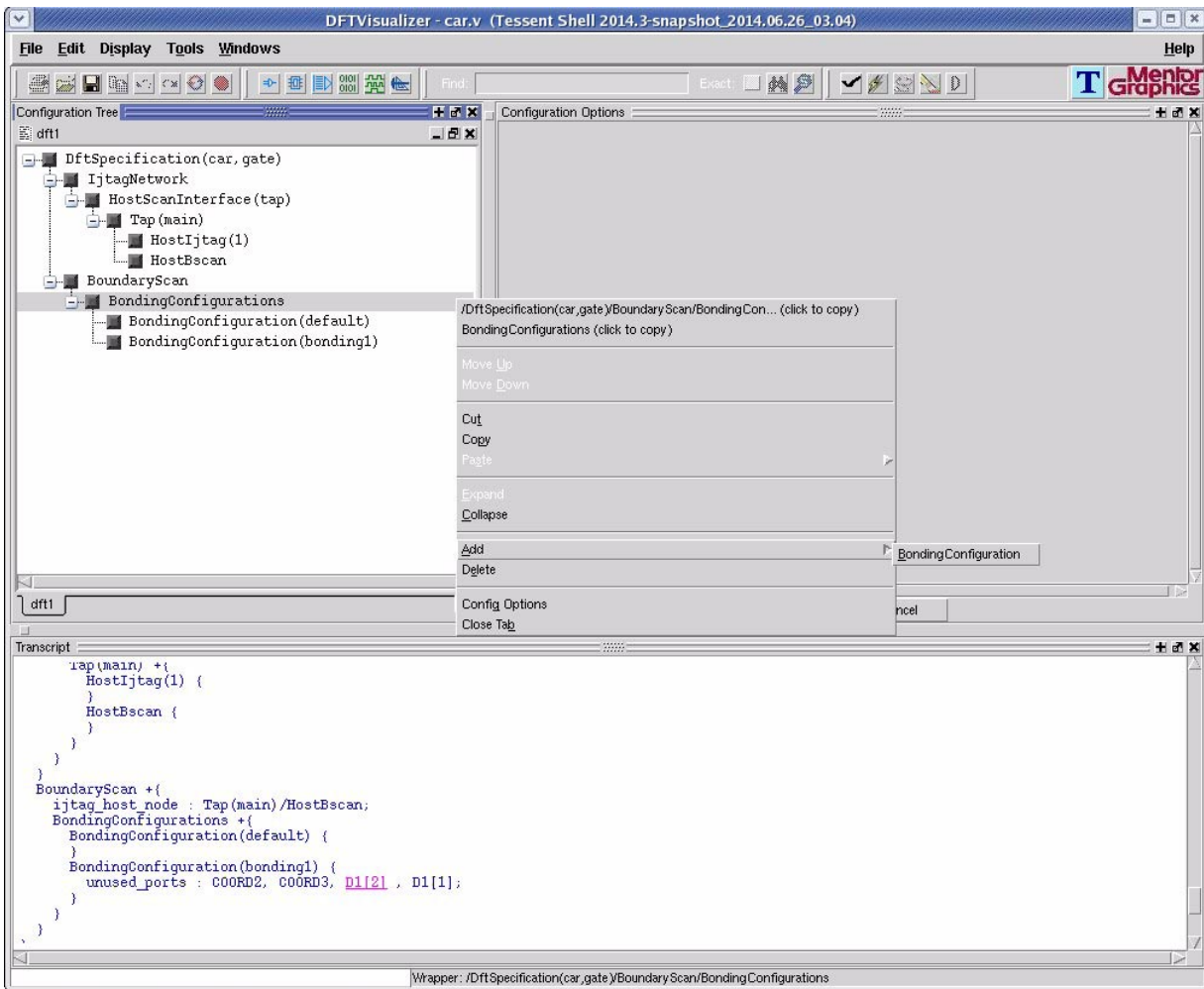
The following example shows how to add multiple package bonding configurations using the GUI and, after edits are made, how to read the configurations back into the dofile using the `read_config_data` command.

```
> set dft_spec [create_dft_specification]
> report_config_data $dft_spec

DftSpecification(car,gate) {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : TCK;
        trst : TRST;
        tms : TMS;
        tdi : TDI;
        tdo : TDO;
      }
      Tap(main) {
        HostIjtag(1) {
        }
        HostBscan {
        }
      }
    }
  }
  BoundaryScan {
    ijtag_host_node : Tap(main)/HostBscan;
  }
}

> display_specification //Opens DFTVisualizer with the DFT specification
                        //displayed
```

Figure 2-5. GUI to Edit DFT Specification



Using DFTVisualizer, make the necessary edits:

1. In the Configuration Tree pane, right-click the BoundaryScan wrapper and select Add > BondingConfigurations from the menu.
2. Right-click the BondingConfigurations wrapper and select Add > BondingConfiguration from the menu.
3. Click the BondingConfiguration wrapper to show the Configuration Options pane.
4. Add the port to be excluded by typing in the port name manually in the unused_ports field.
5. To exclude another port, click the  button and type in another port name. Repeat as necessary.
6. Click Apply to update the GUI as well as the DFT specification in memory.

7. Repeat these steps to add other modifications. In the example, two BondingConfiguration wrappers are added.
8. To see the resulting configuration, use the report_config_data command.

```
> report_config_data $dft_spec
```

```
DftSpecification(car,gate) +{
  IjtagNetwork +{
    HostScanInterface(tap) +{
      Interface {
        tck : TCK;
        trst : TRST;
        tms : TMS;
        tdi : TDI;
        tdo : TDO;
      }
      Tap(main) +{
        HostIjtag(1) {
        }
        HostBscan {
        }
      }
    }
  }
  BoundaryScan {
    ijtag_host_node : Tap(main)/HostBscan;
    BondingConfigurations {
      BondingConfiguration(default) {
      }
      BondingConfiguration(bonding1) {
        unused_ports : COORD2, COORD3, D1[2], D1[1], EN0, D1[0], D1[3];
      }
    }
  }
}
```

Using read_config_data, you can effectively cut and paste the manually entered DFTVisualizer edits into the dofile as shown in the example below. Then for subsequent runs, the configuration edits will already be present in the dofile, making the process repeatable via scripts.

```
read_config_data -in $dft_spec/BoundaryScan -from_string {
  BondingConfigurations +{
    BondingConfiguration(default) {
    }
    BondingConfiguration(bonding1) {
      unused_ports : COORD2, COORD3, D1[2], D1[1], EN0, D1[0], D1[3];
    }
  }
}
```

Note



If you want the edits you made with the GUI to be repeatable and reusable via scripts, read them in your Tcl script or dofile using the read_config_data command.

Method 2: Modify the DFT specification in memory

An alternative method is to modify the DFT specification with the [add_config_element](#) and [set_config_value](#) commands. With this method, the dofile will already contain the commands and the modifications introduced, making the process repeatable.

Example 1

The following example adds multiple package bonding configurations for Tessent BoundaryScan. The `report_config_data` command shows the DFT specification created.

```
> set spec [create_dft_specification]
> report_config_data $spec

DftSpecification(car,gate) {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : TCK;
        trst : TRST;
        tms : TMS;
        tdi : TDI;
        tdo : TDO;
      }
      Tap(main) {
        HostIjtag(1) {
        }
        HostBscan {
        }
      }
    }
  }
  BoundaryScan {
    ijtag_host_node : Tap(main)/HostBscan;
  }
}
```

You can add the necessary BondingConfiguration wrappers and then use the `report_config_data` to see how the DFT specification was updated.

```
> add_config_element $spec/BoundaryScan/BondingConfigurations
> add_config_element $spec/BoundaryScan/BondingConfigurations/\
  BondingConfiguration(default)

> add_config_element $spec/BoundaryScan/BondingConfigurations/\
  BondingConfiguration(bonding1)

> set_config_value $spec/BoundaryScan/BondingConfigurations/\
  BondingConfiguration(bonding1)/\
  unused_ports {COORD2 COORD3 D1[2] D1[1] EN0 D1[0] D1[3]}

> report_config_data
```

```
DftSpecification(car,gate) +{
  IjtagNetwork +{
    HostScanInterface(tap) +{
      Interface {
        tck : TCK;
        trst : TRST;
        tms : TMS;
        tdi : TDI;
        tdo : TDO;
      }
      Tap(main) +{
        HostIjtag(1) {
        }
        HostBscan {
        }
      }
    }
  }
  BoundaryScan {
    ijtag_host_node : Tap(main)/HostBscan;
    BondingConfigurations {
      BondingConfiguration(default) {
      }
      BondingConfiguration(bonding1) {
        unused_ports : COORD2, COORD3, D1[2], D1[1], EN0, D1[0], D1[3];
      }
    }
  }
}
```

Example 2

The following example modifies the DFT specification using the editing commands, including the use of the [delete_config_element](#) command.

```
> set spec [create_dft_specification]
> report_config_data $spec
```

```
DftSpecification(cpu_top,rtl) {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : tck_p;
        trst : trst_p;
        tms : tms_p;
        tdi : tdi_p;
        tdo : tdo_p;
      }
      Tap(main) {
        HostIjtag(1) {
        }
        HostBscan {
        }
      }
    }
  }
}
```

```

    }
    BoundaryScan {
        ijtag_host_node : Tap(main)/HostBscan;
    }
}

```

You can add the necessary BondingConfiguration wrappers and then use the report_config_data to see how the DFT Specification was updated.

```

> add_config_element $spec/BoundaryScan/BondingConfigurations
> add_config_element $spec/BoundaryScan/BondingConfigurations/\
    BondingConfiguration(standard)

> add_config_element $spec/BoundaryScan/BondingConfigurations/\
    BondingConfiguration(package1)

> set_config_value $spec/BoundaryScan/BondingConfigurations/\
    BondingConfiguration(package1)/enable_signal cpu_top/cs

> set_config_value $spec/BoundaryScan/BondingConfigurations/\
    BondingConfiguration(package1)/unused_ports {D2, D3}

> delete_config_element enable_signal -in_wrapper $spec/BoundaryScan/\
    BondingConfigurations/BondingConfiguration(package1)

> delete_config_element unused_ports -in_wrapper $spec/BoundaryScan/\
    BondingConfigurations/BondingConfiguration(package1)

> set_config_value $spec/BoundaryScan/BondingConfigurations/\
    BondingConfiguration(package1)/\
    unused_ports {D1[2] D1[1] D1[0] D1[3]}

> report_config_data

> set spec [create_dft_specification]
> report_config_data $spec

DftSpecification(cpu_top,rtl) +{
    IjtagNetwork +{
        HostScanInterface(tap) +{
            Interface {
                tck : tck_p;
                trst : trst_p;
                tms : tms_p;
                tdi : tdi_p;
                tdo : tdo_p;
            }
            Tap(main) +{
                HostIjtag(1) {
                }
                HostBscan {
                }
            }
        }
    }
}
BoundaryScan {
    ijtag_host_node : Tap(main)/HostBscan;
}

```

```
BondingConfigurations {  
  BondingConfiguration(standard) {  
  }  
  BondingConfiguration(package1) {  
    unused_ports : D1[2], D1[1], D1[0], D1[3];  
  }  
}  
}
```

Validate the DFT Specification

In this optional step, you can validate the edits you made to the DFT specification to make sure no errors exist before you proceed to the next step.

Example

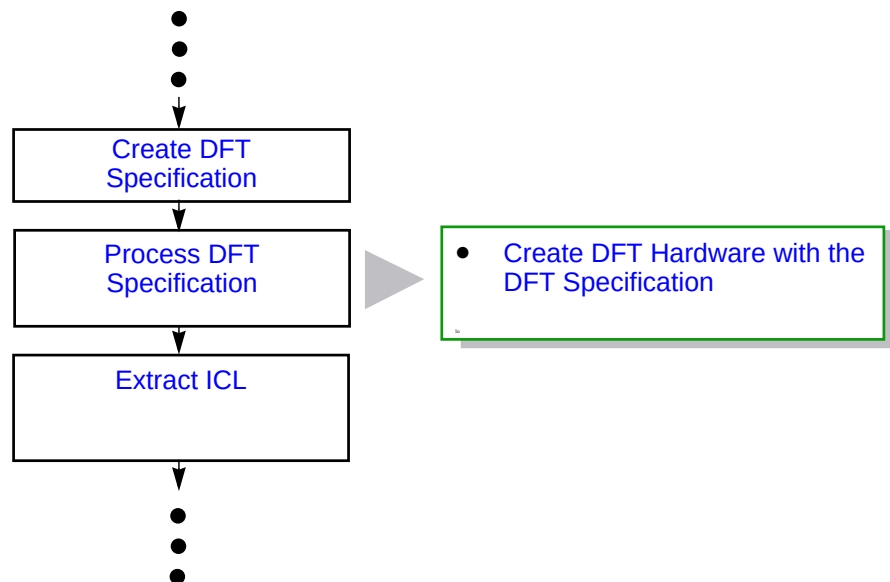
The following example validates your DFT specification.

```
process_dft_specification -validate_only
```

Process DFT Specification

The next step is to process the DFT specification that was created, edited, and validated in the previous step. This step creates and inserts the hardware for all the components that are in the DFT specification.

Figure 2-6. Process DFT Specification



Create DFT Hardware with the DFT Specification

Use the `process_dft_specification` command to generate and insert into the design all DFT hardware requested with the DFT specification. For Tessent BoundaryScan, the TAP controller and boundary-scan cells are inserted. JTAG is used because boundary scan connects to the TAP, and the TAP is an JTAG node.

Example 1

The following example generates and inserts into the design the hardware requested with the DFT specification.

```
process_dft_specification
```

Example 2

The following example generates the hardware requested with the DFT specification but does not insert the hardware into the design.

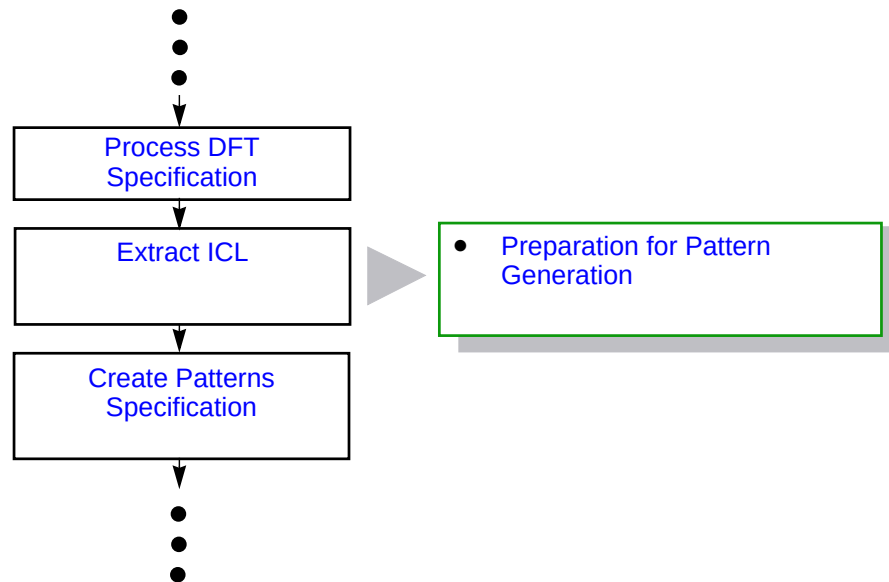
```
process_dft_specification -no_insertion
```

Extract ICL

The Extract ICL step verifies the proper connectivity of the ICL modules that were inserted with the `process_dft_specification` command. If no DRC violations are detected, the top-level ICL description is extracted.

Downstream tools use the top-level ICL description for creating patterns. You can use the [open_visualizer](#) command to debug any ICL extraction DRC violations that are reported.

Figure 2-7. Extract ICL



Preparation for Pattern Generation

In this step, use the `extract_icl` command to find all modules (both Tessent instruments and non-Mentor Graphics instruments) with their associated ICL modules. If no DRC violations are detected, the ICL description of the root design is extracted.

The root of the design was specified with the `set_current_design` command during design elaboration in the [Design Loading](#) step. The [Create Patterns Specification](#) and [Process Patterns Specification](#) steps use the ICL description that was created for the root of the design. You can use the `open_visualizer` command to debug ICL extraction DRC violations. Refer to the “[DFTVisualizer](#)” chapter in the *Tessent Shell User’s Manual*.

Example

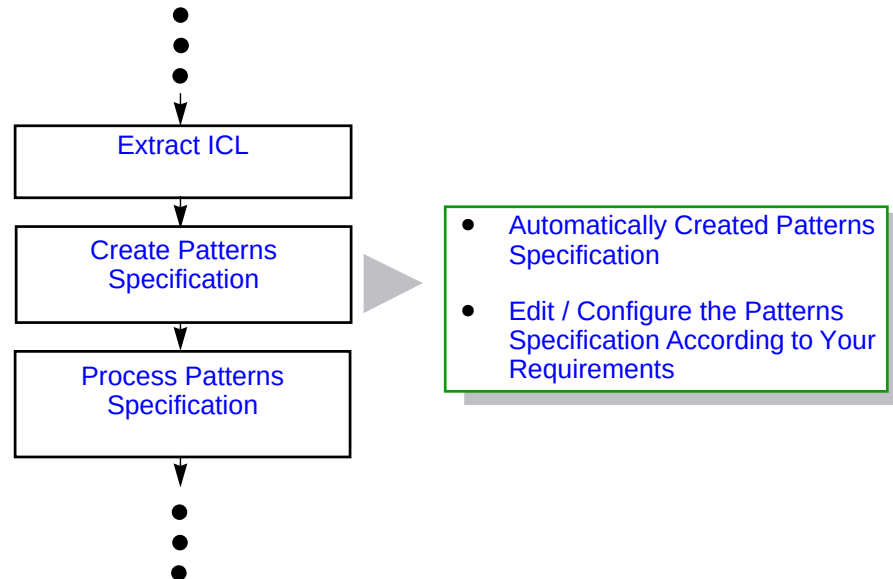
The following example extracts all ICL modules to the root of the design.

```
extract_icl
```

Create Patterns Specification

The Create Patterns Specification step creates the default patterns specification. The patterns specification is a configuration file that tells you what tests are being created. You can edit or configure the default patterns specification to generate the desired patterns specification.

Figure 2-8. Create Patterns Specification



Automatically Created Patterns Specification

The default patterns specification, which is created with the `create_patterns_specification` command, is only stored in memory.

To see the specification, use the `report_config_data` command.

Example

The following example creates a default patterns specification and stores the specification in a variable called `pat_spec` and then uses this variable to report the patterns specification in memory. The use of the Tcl variable is not necessary and is only used for convenience.

```
set pat_spec [create_patterns_specification]
report_config_data $pat_spec
```

Edit / Configure the Patterns Specification According to Your Requirements

Use one of the following methods to edit or create a patterns specification according to your requirements. Typically you do not need to edit the Signoff patterns specification; only the Manufacturing patterns specification may need editing based on your requirements.

Method 1: Edit the patterns specification in memory

This is the preferred method because as you edit the patterns specification in memory, the commands that are used are specified in the Tcl or dofile and, therefore, the edits are repeatable for the next iteration by using only scripts.

Example

The following example shows that the default value for `tester_period` is 100ns and can be changed by editing the patterns specification in memory.

```
set pat_spec [create_patterns_specification]
report_config_data $pat_spec
set_config_value $pat_spec/Patterns(JtagBscanPatterns)/tester_period 50ns
```

Method 2: Write out the patterns specification, edit the file, and read the file back in

This method may be easier, but keep in mind that every time the master Tcl script or dofile runs, the patterns specification that is written out will overwrite your edits. To make sure your edits are reusable and repeatable using scripts, make a copy of the patterns specification and then edit the specification before reading it back in.

Example

The following example writes the patterns specification into a file called `bonding1_config.pat_spec`. After making the desired edits in a copy of this file, this file is read back in. Note that the file that is written out is different from the edited file that is read back in.

```
create_patterns_specification
report_config_data
write_config_data bonding1_config.pat_spec -wrappers
    PatternsSpecification(car,signoff)
read_config_data bonding1_config_edited.pat_spec
report_config_data
```

When manually editing the patterns specification, it is highly recommended that you validate the specification as shown in the following example.

```
process_patterns_specification -validate_only
```

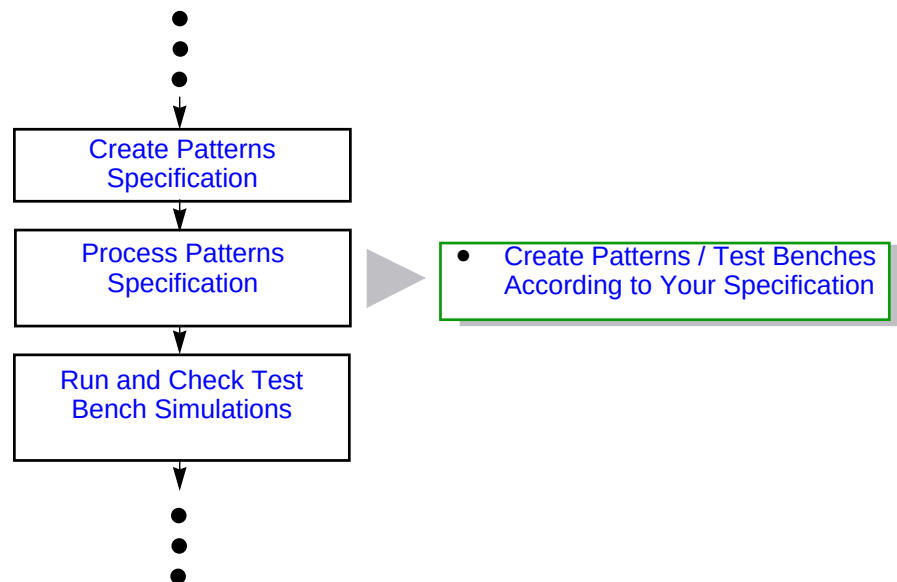
Method 3: Use the GUI to edit the Patterns Specification

The third option is to use the `display_specification` command and edit the patterns specification through the GUI.

Process Patterns Specification

The Process Patterns Specification creates the patterns and test benches.

Figure 2-9. Process Patterns Specification



Create Patterns / Test Benches According to Your Specification

In this step, you create the patterns or test benches according to either the default patterns specification or to the edited patterns specification that you created in the previous step.

Example

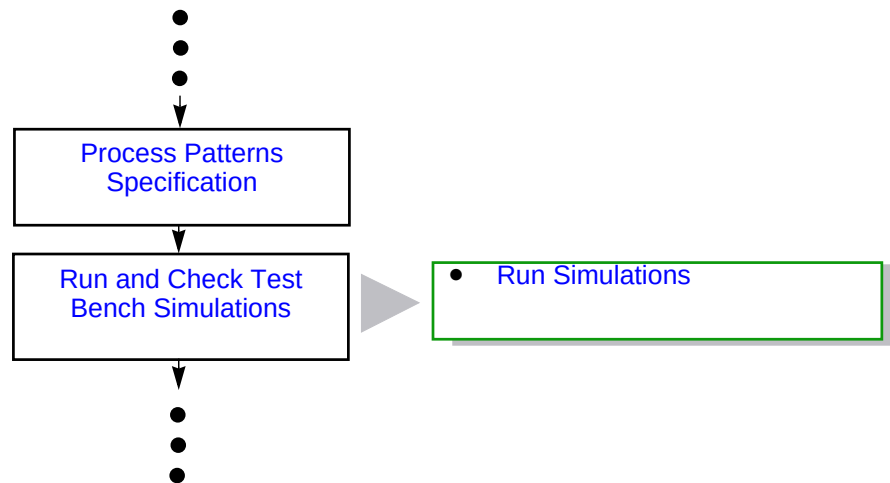
The following example generates the test benches.

```
process_patterns_specification
```

Run and Check Test Bench Simulations

The Run and Check Test Bench Simulations step is the last step in the Tessent BoundaryScan insertion using Tessent Shell. In this step, you run simulations of the boundary scan verification and then check the results.

Figure 2-10. Run and Check Test Bench Simulations



Run Simulations

Use the `run_testbench_simulations` command to invoke a simulation manager to run a set of simulation test benches.

The `run_testbench_simulations` command compiles and simulates test benches, generated for the TAP and boundary scan from the `process_patterns_specification` command, that are located at `tsdb_outdir/patterns/<design>.patterns_signoff`.

For a detailed description of the `run_testbench_simulations` command and its usage, see the *Tessent Shell Reference Manual*.

Example

The following example runs simulations of all patterns defined in the PatternsSpecification.

```
set_simulation_library_sources -y ../library/verilog \  
-v ../library/pad_cells.v  
run_testbench_simulations
```

Check Results

Use the `check_testbench_simulations` command to check the status of the simulations that were previously launched by the `run_testbench_simulations` command.

For a detailed description of the `check_testbench_simulations` command and its usage, see the *Tessent Shell Reference Manual*.

Example

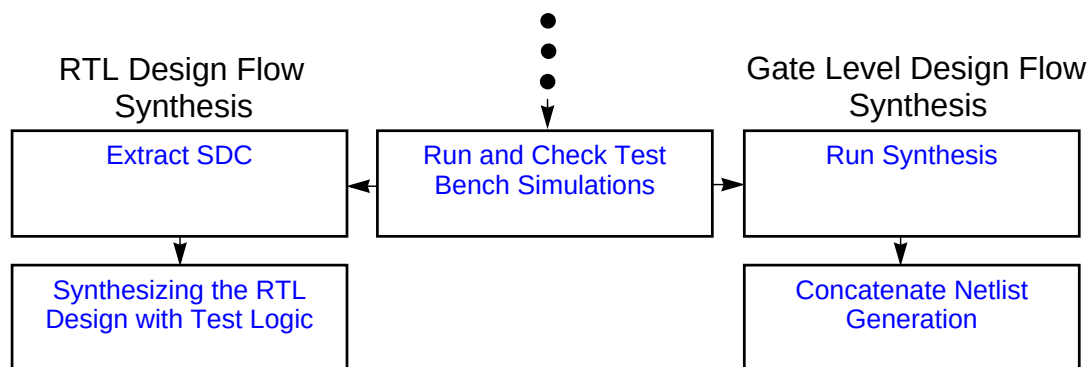
The following example checks the simulation results for errors.

```
check_testbench_simulations -report_status
```

Test Logic Synthesis

The test logic synthesis process is different when handling a RTL or gatelevel designs. The following sections outline the different options.

Figure 2-11. Test Logic Synthesis



RTL Design Flow Synthesis

The RTL synthesis flow that integrates the MBIST RTL and associated test logic with the design RTL is not a fully automated flow. The following sections outline the process, which have several manual steps.

Extract SDC

The `extract_sdc` command creates an SDC file that can be used for synthesis.

The command requires ICL in order to create the SDC, so is typically run after [extract_icl](#), which needs to be run before [create_patterns_specification](#).

When [extract_sdc](#) is run on physical block containing sub-blocks, the constraints are generated for the physical module as well as the sub-blocks

The created SDC is composed of several procedures that can be integrated into a design synthesis script. The names and function of the procedures are outlined below:

- `tessent_set_default_variables`

This proc defines the default value of the variables used in instrument timing constraints. It is provided as a template. The user should override the values of these variables to correspond to their setup. For example, the values of the array "tessent_clock_mapping" should be overwritten to the user's chosen names for the functional clocks.

- `tessent_create_functional_clocks`

This proc defines the functional clocks used by the instrument constraints. This proc is not typically called since most users will define their functional clocks in their own timing scripts.

- `tessent_constrain_<design_name>_<mode>`

This proc defines the constraints for instruments whose constraints can be merged with other instrument's constraints. The mode will typically be modal or non_modal.

- `tessent_constrain_<design_name>_<instrument>_<mode>`

These procs define the constraints for each instrument in its given mode. The mode will typically be modal or non_modal.

Example

The following example creates SDC for a design and can be run at the physical_block or top design level.

```
extract_sdc
```

Synthesizing the RTL Design with Test Logic

This is currently a semi-manual process and requires the user to gather both the generated test logic RTL as well as the edited design RTL.

The `<design_name>.design_source_dictionary` located in the in the TSBD (`tsdb_outdir/dft_inserted_designs/<design_name>_<design_id>_rtl.dft_inserted_design`) points to all files for the test inserted design and can be used to create a full design synthesis script that also combines the generated logic test, as well as the SDC created by [extract_icl](#).

Further automating is planned to simplify this process and will be available future releases.

Gate Level Design Flow Synthesis

The gate level design flow synthesis is a fully automated flow and only requires the `run_synthesis` command to synthesize the test logic and integrate into the design,

Run Synthesis

The `run_synthesis` command only synthesizes test logic RTL contained within the TSBD.

When creating and inserting memory BIST, boundary scan or IJAG logic, the generated RTL is automatically written to the TSDB during [process_dft_specification](#).

The `run_synthesis` command to invokes a synthesis manager to perform synthesis of the test logic RTL

For a detailed description of the [run_synthesis](#) command and its usage, refer to the *Tessent Shell Reference Manual*.

Example

The following example performs synthesis for a design and can be run at the `physical_block` or top design level.

```
run_synthesis
```

Concatenate Netlist Generation

When `run_synthesis` completes successfully, a concatenated netlist of the design which contains the synthesized test logic and modified design modules will automatically be created and placed in the `dft_inserted_designs` directory of the TSDB.

Chapter 3

Boundary Scan Specific Topics

Topics within this chapter cover a variety of subjects that describe cell library requirements as well as insertion of boundary scan cells in different design scenarios.

Pad Cell Library	43
Inserting Tessent Boundary Scan on a Custom or Pre-Existing TAP Controller	45
Sharing TAP Ports Between a Pre-Existing TAP and Tessent TAP	46
AC JTAG	48
Embedded Boundary Scan (EBscan)	48
Adding a Test Data Register (TDR) to the TAP Controller	49
BSDL-Only Flow	52
Dividing Boundary Scan for Logic Test	54
Multiple Bonding Configurations	56
Custom Boundary Scan Cells	59
Debugging Failing JtagBscan Simulations	61

Pad Cell Library

A Tessent cell library for pad cells is needed for inserting TAP and BoundaryScan using Tessent tools. The Tessent cell library is an integrated library that contains a functional description for simulation as well as attributes for test logic insertion for each cell.

For further information on how to generate a Tessent cell library, refer to the [Tessent Cell Library Manual](#).

The following sections describe alternate methods if you already have a *pad.library* file, which contains pad IO cell descriptions and verilog simulation models for the pad cells.

Using *pad.library* and Verilog Simulation Models for the Pad Cells

Use this procedure if you have pad cell verilog simulation models and a *pad.library* file that contains pad cell IO descriptions.

Prerequisites

- *pad.library* file containing pad IO cell descriptions.

- Verilog models for the pad IO cells describing the functional behavior.
- Tessent Shell is invoked and ready to begin the [Read the Libraries](#) step of [Design Loading](#) as shown [Figure 2-2](#).

Procedure

1. Read the pad.library file from the appropriate directory, as shown in the following example.

```
SETUP>read_cell_library ../library/pad.library
```

2. Read the verilog models for the pad cells from the appropriate directory, as shown in the following example.

```
SETUP>read_verilog ../library/verilog/pads.v
```

Results

The attributes of the pad cells described within the *pad.library* file will be applied on top of the verilog description of the pad cells that were read in using the read_verilog command.

Creating a Tessent Cell Library from *pad.library* and Verilog Simulation Models

As outlined in the previous section, it is shown that the Tessent Cell Library is not needed if you have a *pad.library* file and verilog simulation models. The procedure outlined in this section describes an optional way of creating a Tessent Cell Library utilizing the *pad.library* and verilog simulation models.

Prerequisites

- *pad.library* file containing pad IO cell descriptions.
- Verilog models for the pad IO cells describing the functional behavior.

Procedure

1. Invoke the [LibComp](#) tool from the unix shell prompt to compile a libcomp.atpglib file from the pad cell verilog simulation models. The first parameter passed is the path to the verilog simulation models for the pad cells.

```
UNIX>libcomp pads.v -dofile
```

Note



You do not need to provide any do files with the -dofile switch.

A libcomp.atpglib file will be created for all the cells present in the pads.v file. Ensure that any includes using 'include in the pads.v file are properly read in.

2. Invoke Tessent Shell and optionally create a log file for reference.

```
UNIX>tessent -shell -log create_cell_lib.log
```

3. Read the pad.library and the libcomp.atpglib file created in step1 using read_cell_library at the Tessent Shell prompt.

```
SETUP>read_cell_library pad.library  
SETUP>read_cell_library libcomp.atpglib
```

4. Write the new Tessent Cell Library using write_cell_library command.

```
SETUP>write_cell_library pads.tcell_lib
```

Results

The created Tessent Cell Library can be read into Tessent Shell using the read_cell_library command.

If LibComp is unable to translate a cell in the pads.v file into an atpg.lib model, an empty blackbox model is created for that cell. In this situation, the translation of these cells into the Tessent Cell Library as functional library elements is not completed and they will have to be manually edited into valid functional models.

Related Topics

[read_cell_library](#)

[write_cell_library](#)

Inserting Tessent Boundary Scan on a Custom or Pre-Existing TAP Controller

The Tessent Shell design flow natively supports a custom or pre-existing TAP controller connection to Boundary Scan cells. Custom or pre-existing TAP controllers can also be connected to other controllers, such as memory BIST or hybrid TestKompress/LBIST.

In order for Tessent Shell to work with the pre-existing TAP controller, an ICL(Instrument Connectivity Language) description for the TAP controller needs to be created. For further information please refer to the “[Instrument Connectivity Language\(ICL\)](#)” section in the *Tessent Shell Reference Manual*. Additionally, the pre-existing TAP controller must have the port functions described in the “Requirements on a TAP to be usable for BoundaryScan” section within the [DftSpecification/BoundaryScan](#) wrapper description in the *Tessent Shell Reference Manual*.

Prerequisites

- An ICL description for a the custom or pre-existing TAP controller.
- A TAP controller that meets the requirements outlined in the [DftSpecification/BoundaryScan](#) wrapper description.

Procedure

1. Read in the verilog model and the ICL description of the pre-existing or custom TAP at the “[Read the Design](#)” step within the “[Design Loading](#)” portion of the design flow.

```
SETUP>read_verilog design/my_custom_TAP.v  
SETUP>read_icl design/my_custom_TAP.icl
```

If there is only a single HostBscan interface defined, the boundary scan chain will be connected to the pre-existing TAP controller that is read in if all the requirements of the TAP controller are met.

2. If the pre-existing TAP controller has more than one HostBscan interface, you can explicitly specify which interface the boundary scan chain will be connected to using the following:

```
ANALYSIS>create_dft_specification -existing_bscan_host_scan_in \  
my_custom_TAP_INST/fromBscan1
```

my_custom_TAP_INST in the example above is the instance name of the pre-existing custom TAP controller in the design and fromBscan1 is the port on the pre-existing TAP where the boundary scan chain is to be connected.

Related Topics

[create_dft_specification](#)[read_icl](#)[read_verilog](#)

Sharing TAP Ports Between a Pre-Existing TAP and Tessent TAP

TAP pins can be shared between a pre-existing TAP and the Tessent TAP controller.

There may be situations where the pre-existing custom TAP controller does not meet the requirements described in the [DftSpecification/BoundaryScan](#) wrapper description in the *Tessent Shell Reference Manual* and cannot be used for boundary scan. Perhaps there is a design requirement to not disturb the pre-existing TAP controller and it is desired to add the Tessent TAP controller to connect to the boundary scan chain. In either scenario, you can share the existing TAP pins between the pre-existing TAP controller and the Tessent TAP that is to be inserted as part of the Tessent Boundary Scan insertion flow using this procedure.

Prerequisites

- The IEEE 1687 standard requires that the state of a TAP controller should not change when its scan path is not selected. When implementing the methodology outlined in this procedure, the pre-existing TAP will have its own TMS gating. When the Tessent TAP is selected by its specified Compliance Enable signals, the pre-existing TAP is gated off, preserving it's state.

Procedure

1. Specify the TAP pins during the “[Specify and Verify DFT Requirements](#)” section of the design flow, as shown in the following example:

```
SETUP>set_attribute_value tck_p -name function -value tck
SETUP>set_attribute_value tdi_p -name function -value tdi
SETUP>set_attribute_value tms_p -name function -value tms
SETUP>set_attribute_value trst_p -name function -value trst
SETUP>set_attribute_value tdo_p -name function -value tdo
```

2. Specify the Compliance Enable (CE) pins:

```
ANALYSIS>create_dft_specification \
    -active_high_compliance_enables CEO \\<pin_port_list>
    -active_low_compliance_enables CE1 \\<pin_port_list>
```

When specifying the Compliance enable pins with [create_dft_specification](#), the `active_high_compliance_enables` and `active_low_compliance_enables` properties within the [HostScanInterface/Interface](#) are filled with the named ports and they are also added to the BoundaryScan/ [BoundaryScanCellOptions](#) wrapper as `compliance_enable1` and `compliance_enable0`.

The Tessent TAP will be selected during boundary scan tests when the CE0 port is “1” and the CE1 port is “0”. These values will automatically be set during boundary scan test simulations and will be documented in the generated BSDL file.

A module named “CE decoder” will be created that muxes the TDO signal and gates the TMS signal to the Tessent TAP based on the compliance enable signal values. If we are inserting boundary scan, then `<pin_port_list>` should list ports as they need to be identified in the Boundary Scan Description Language (BSDL) file. If an internal pin is provided in the list, then a warning is given that the BSDL file will not be IEEE Std 1149.1 compliant.

Related Topics

[set_attribute_value](#)

[HostScanInterface/Interface](#)

[create_dft_specification](#)

[BoundaryScanCellOptions](#)

AC JTAG

If there are any AC JTAG pad IO cells present in the design, then a Tessent Cell Library describing them needs to be created and read in during the “Design Loading” section of the design flow.

For more information, refer to Chapter 2 “[Library Model Creation](#)” of the *Tessent Cell Library Manual*.

If an old *pad.library* file is available and verilog models are present, then the procedure described in “[Using pad.library and Verilog Simulation Models for the Pad Cells](#)” can be used.

Embedded Boundary Scan (EBscan)

The Embedded Boundary Scan flow is typically used where the pad IO cells are present inside a module that is either a sub-block or a physical region. If the boundary scan cells need to be present within this sub-block or the physical region, then the embedded boundary scan flow is used.

To implement this flow, the command [set_boundary_scan_port_options](#) is used to specify the ports that require embedded boundary scan cells. This process is performed in the [Specify DFT Specification Requirements](#) step after [Design Loading](#) completed. The following procedure shows an example of how this is done for both sub-block and physical region implementations.

Prerequisites

- [Design Loading](#) steps have been completed and the design is loaded and elaborated.

Procedure

1. Start the [Specify and Verify DFT Requirements](#) step by beginning to define the DFT specification requirements.

```
## Begin Specify and Verify DFT Requirements step
SETUP>set_dft_specification_requirements -boundary_scan on
```

2. Specify at what level boundary scan is to be inserted.

For sub-block implementation:

```
SETUP>set_design_level sub_block
```

For physical region implementation:

```
SETUP>set_design_level physical_region
```

3. Specify the list of pad IO ports that need embedded boundary scan cells by using the [set_boundary_scan_port_options](#) command.

```
## The following will insert embedded boundary scan cells
```



```
SETUP>set_boundary_scan_port_options -pad_io_ports [list in1 \
in_diff_p in_diff_n out1 out_diff_p out_diff_n clk A Y]
```

4. Use the [add_input_constraints](#) command to specify the constants that are required for the pad IO to operate.

Note

Some of the pad IO constants that are required may come from the next higher level, either through test setup or other initialization setup.

```
##Constraints specified for pins where the value comes from the next
level
SETUP>add_input_constraints vss -C0
SETUP>add_input_constraints Ten3[2] -C0
SETUP>add_input_constraints Ten3[1] -C0
SETUP>add_input_constraints Ten4[2] -C0
SETUP>add_input_constraints vdd -C1
SETUP>add_input_constraints Ten1 -C1
SETUP>add_input_constraints Ten2 -C1
SETUP>add_input_constraints Ten3[0] -C1
SETUP>add_input_constraints Ten4[1] -C1
```

5. Run [check_design_rules](#) to verify the implementation and then continue with the rest of the design flow.

```
##Running DRC
SETUP>check_design_rules
```

Related Topics

[add_input_constraints](#)

[set_dft_specification_requirements](#)

[set_boundary_scan_port_options](#)

Adding a Test Data Register (TDR) to the TAP Controller

User defined bits for test control and status monitoring can be added through the creation of a Test Data Register (TDR) that is inserted and connected to the TAP controller.

The [Tdr/DataInPorts](#) wrapper specifies the number of data-in ports to create for test status monitoring on the TDR, the naming of the ports, and the connections to make to them. The [Tdr/DataOutPorts](#) wrapper specifies the number of data-out ports to create for test control on the TDR, the naming of the ports, and the connections to make to them. For more information, refer to the [Tdr](#) section in the *Tessent Shell Reference Manual*.

The procedure below provides an example for how one can create a default DFT specification and manually add a TAP using the DFTVisualizer GUI. A method is shown for copying the

DFT specification, inserting the TDR without using the DFTVisualizer GUI and reading the DFT Specification back into memory. This is used to facilitate creation of dofiles to make the process repeatable without having to manually edit it within DFTVisualizer.

Prerequisites

- [Design Loading](#) steps have been completed and the design is loaded and elaborated.
- [Specify and Verify DFT Requirements](#) steps are completed.

Procedure

1. Create a DFT specification and also direct the output to a Tcl variable:

```
##To create a dft specification  
set spec [create_dft_specification]
```

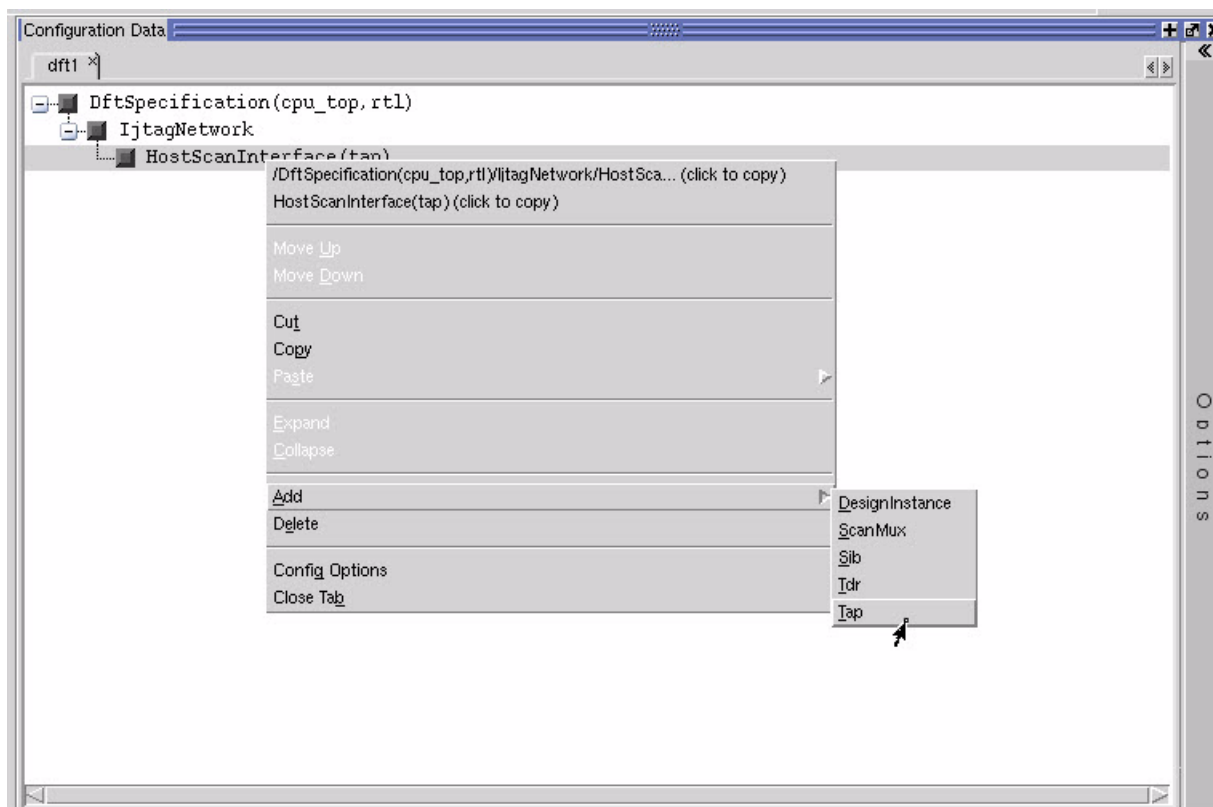
2. Invoke DFTVisualizer and display the Configuration Data Window and Configuration Options pane.

```
display_specification
```

3. Within DFTVisualizer, add in the TAP structure by selecting and highlighting the desired location within the DftSpecification tree, right clicking and selecting **Add>Tap**, as shown in [Figure 3-1](#). The path this example implements to the TAP named “main” is:

```
/IjtagNetwork/HostScanInterface(tap)/Tap(main)
```

Figure 3-1. Adding a TAP with DFTVisualizer



4. In the Console window, use `report_config_data` to list the current structure.

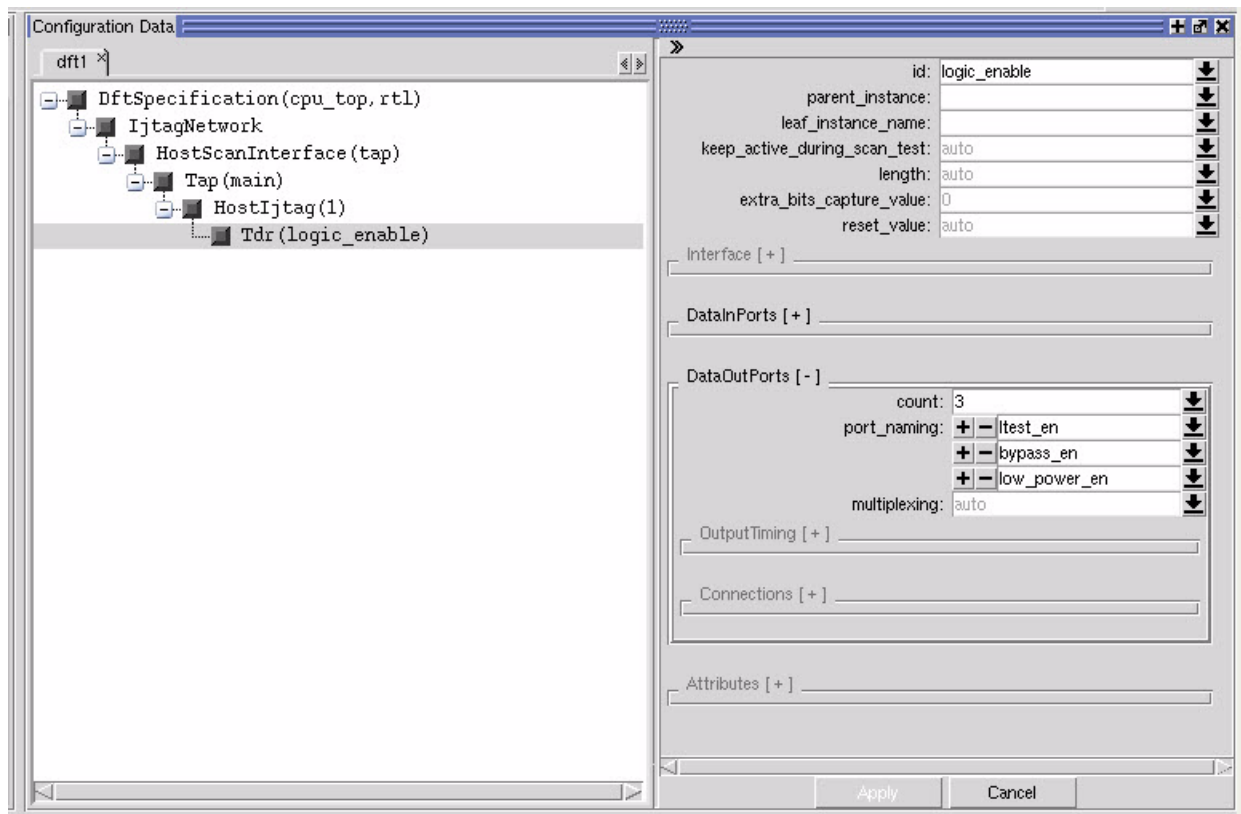
```
report_config_data $spec
```

5. Once the configuration is reported, you can cut and paste this information into the dofile, reading the DFTVisualizer edits implementing the TAP and the manually entered TDR, by using the `read_config_data` command as shown in the example below. Then, for subsequent runs, the configuration edits will already be present in the dofile, making the process repeatable via scripts.

```
read_config_data -in_wrapper
$spec/IjtagNetwork/HostScanInterface(tap)/Tap(main)/HostIjtag(1) -
from_string {
    Tdr(logic_enable) {
        DataOutPorts +{
            count          : 3;
            port_naming    : ltest_en, bypass_en, low_power_en;
        }
    }
}
```

6. The TDR that was added can be inspected within DFTVisualizer as shown in [Figure 3-2](#).

Figure 3-2. Viewing the Added TDR in DFTVisualizer



7. Create the hardware from the DFT specification;

```
process_dft_specification
```

Related Topics

[create_dft_specification](#)

[display_specification](#)

[report_config_data](#)

[read_config_data](#)

BSDL-Only Flow

This section describes how you can generate boundary scan patterns in Tessent Shell using a Boundary Scan Definition Language (BSDL) file created outside of Tessent Boundary Scan tools.

Prerequisites

- A valid BSDL file.

Procedure

1. The configuration data to specify the patterns to be applied to DFT components that are inserted into a design is defined within the [BoundaryScan](#) wrapper in the [PatternsSpecification](#). An example of the wrapper contents is shown in [Figure 3-3](#). This would be saved as a file, and for this example it will be named `example.patterns_spec_signoff`. The third party BSDL file is specified in the [BoundaryScan](#) wrapper with the `bsdl_file` property.

Figure 3-3. Example PatternsSpecification for Third Party BSDL

```
PatternsSpecification(example,bscan,signoff) {
  Patterns(JtagBscanPatterns) {
    tester_period : 100ns;
    TestStep(JtagBscanTestStep) {
      BoundaryScan {
        bsd_file : EP432P12.bsd;
        RunTest(test_logic_reset) {
        }
        RunTest(inst_reg) {
        }
        RunTest(bypass_reg) {
        }
        RunTest(bscan_reg) {
        }
        RunTest(input) {
        }
        RunTest(sample) {
        }
        RunTest(highz) {
        }
        RunTest(clamp) {
        }
        RunTest(output) {
        }
      }
    }
  }
}
```

For more explanation of the [RunTest](#) wrapper and parameters, refer to the [BoundaryScan](#) topic in the “[Configuration-Based Specification](#)” chapter in the *Tessent Shell Reference Manual*.

2. Create a Tcl file containing the Tessent Shell steps to set the proper context, read in the configuration wrapper and generate the patterns. An example file is shown in [Figure 3-4](#) and will be named `bsdl_only.tcl` for this procedure.

Figure 3-4. Tessent Shell dofile for Third Party BSDL Processing

```
set_context pattern -ijtag -design_id bscan
read_config_data ./example.patterns_spec_signoff
process_pattern_specification
exit
```

3. Invoke Tessent Shell and specify bsd1_only.tcl as the dofile to execute.

```
tessent -shell -dofile bsd1_only.tcl -log bsd1_patterns.log -replace
```

Results

The testbench “JtagBscanPatterns.v” is created in the tsdb_outdir/Patterns/example_bscan.patterns_signoff directory. For details on how the tsdb_outdir directory structure is organized, please refer to “[Tessent Shell Data Base \(TSDB\)](#)” in the *Tessent Shell Reference Manual*.

Related Topics

[BoundaryScan wrapper](#)

[RunTest](#)

[PatternsSpecification wrapper](#)

Dividing Boundary Scan for Logic Test

In a majority of designs, the boundary scan chain needs to be included in the Logic testing portion of the design so the chip boundaries are isolated with a boundary scan chain. For most designs, the single boundary scan chain is too long as there is some type of compression or Built In Self Test (BIST) incorporated to test the logic portion of the design.

The boundary scan chain needs to be divided, or segmented into shorter chain lengths in this situation to match how the functional design flops are stitched into scan chains. The max_segment_length_for_logictest property in the [BoundaryScan](#) wrapper can be implemented to properly segment the boundary scan chain.

The example presented in this section will segment the boundary scan chain so a maximum of 200 flops are present in any segment.

Restrictions and Limitations

Prerequisites

- [Design Loading](#) steps have been completed and the design is loaded and elaborated.
- [DFT Specification requirements](#) and [constraints](#) are defined.

Procedure

1. Run Design Rule Checking (DRC) to ensure all constraints are correct and move Tessent Shell from Setup to Analysis mode if no errors are found.

```
SETUP>check_design_rules
```

2. Create the “DftSpecification(design_name,id)” configuration wrapper and copy the newly created wrapper object to the variable “spec” to customize the specification later.

```
##Create a dft specification
ANALYZE>set spec [create_dft_specification]
```

3. Set the value for the max_segment_length_for_logictest parameter in the BoundaryScan wrapper to 200.

```
ANALYZE>set_config_value \
$spec/BoundaryScan/max_segment_length_for_logictest 200
```

4. Create and insert the hardware for the TAP and boundary scan into the design.

```
ANALYZE>process_dft_specification
```

5. Inspect the results to confirm the boundary scan is now segmented into the smaller chain segments for inclusion with logic testing, and save the output for later reference if desired.

List the currently available instrument dictionaries created by process_dft_specification.

```
ANALYZE>get_instrument_dictionary -list
```

A list will be returned similar to that shown below:

```
DftSpecification LibraryCells RtlCells
mentor::ijtag::DftSpecification
mentor::jtag_bscan::DftSpecification
mentor::memory_bisr
tshell_global
```

List the scan chains and scan_in/scan_out ports created. This also identifies how many scan chains segments were created.

```
ANALYSIS>format_dictionary [get_instrument_dictionary \
mentor::jtag_bscan::DftSpecification logic_test_scan_chains]
```

Optionally, save the output created in step 6 for later reference by using the script below. This saves the output to a file named “logic_test_scan_chains.dictionary”.

```
set fp [open logic_test_scan_chains.dictionary w]
puts $fp "set logic_test_scan_chains {"
puts $fp [format_dictionary [get_instrument_dictionary
mentor::jtag_bscan::DftSpecification logic_test_scan_chains ] ]
puts $fp "}"
close $fp
```

Results

The single boundary scan chain that is connected between the TDI and TDO is now segmented, with muxes added in and controlled by logic_test_enable. The single boundary scan chain connectivity is also maintained.

Related Topics

[create_dft_specification](#)

[set_config_value](#)

[format_dictionary](#)

Multiple Bonding Configurations

This section describes the process flow to add support for multiple package bonding configurations.

When you have multiple package bonding configurations that need to be supported, you will need to edit the DftSpecification wrapper to add the various package options. One way, as demonstrated here, is to use DFTVisualizer to specify the bonding configurations and then read it into the main Tcl or dofile so the process is repeatable without manually editing in DFTVisualizer. You can also edit the DftSpecification as described in “[Method 2: Modify the DFT specification in memory](#)” in Chapter 2.

Prerequisites

- [Design Loading](#) steps have been completed and the design is loaded and elaborated.
- [Specify and Verify DFT Requirements](#) steps have been completed.

Procedure

1. Create a DFT specification and also direct the output to an environment variable. Report out the DFT specification for verification:

```
##To create a dft specification
ANALYSIS>set spec [create_dft_specification]

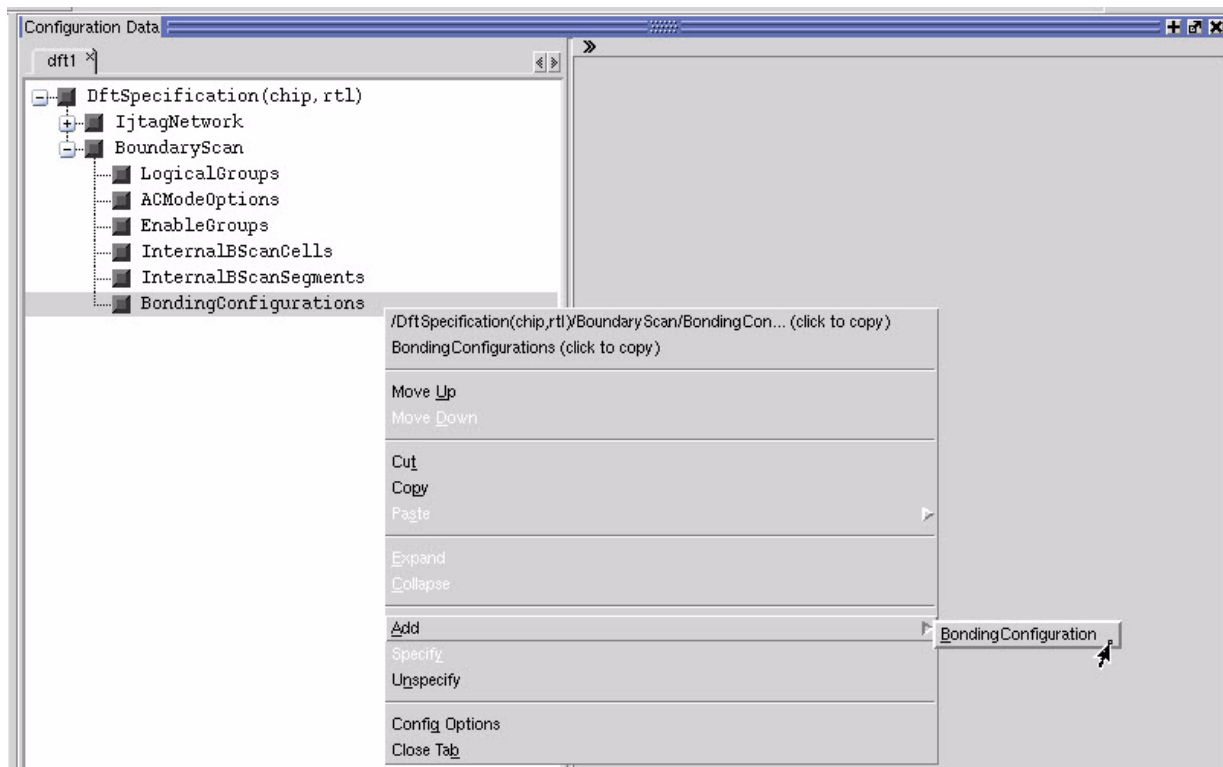
##Report on what dft spec was created
ANALYSIS>report_config_data $spec
```

2. Invoke DFTVisualizer to modify the dft specification so multiple bonding configurations can be added. DFTVisualizer will automatically open the [Configuration Data Window](#) and display the DftSpecification(<design_name>,<design_id>) wrapper configuration when [display_specification](#) is executed.

```
##Invoke DFTVisualizer and display the Configuration Data Window
ANALYSIS>display_specification
```

3. Expand the DftSpecification tree by clicking the  icon to expand DftSpecification/BoundaryScan and display BondingConfigurations under BoundaryScan. Select BondingConfigurations in the tree and right click to display a menu of options available. Select **Add > BondingConfiguration** to add a configuration as shown in [Figure 3-5](#). Repeat as needed for the number of bonding configurations required in your design. Two will be added for this example.

Figure 3-5. Adding Bonding Configurations in DFTVisualizer

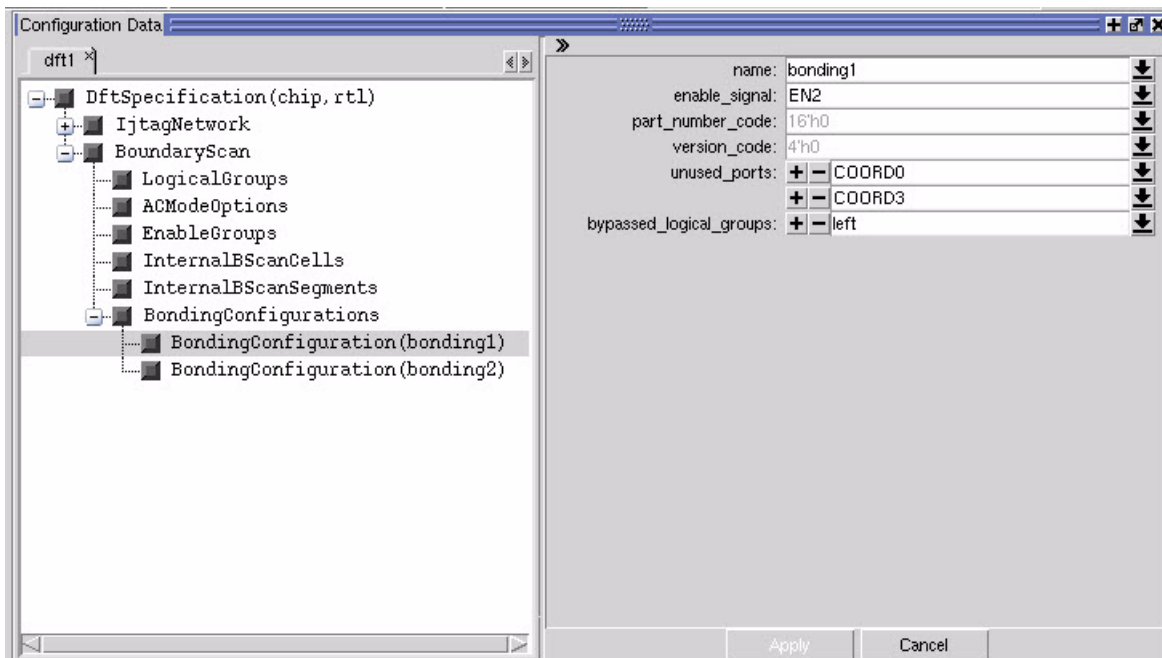


4. Click  at the top of the Options tab in the Configuration Data Window to expand the Configuration Options pane. Select a BondingConfiguration in the tree and fill in the desired options as shown in [Figure 3-6](#). The names are provided by the user and will be bonding1 and bonding2 for this example. Additionally, bonding1 has an enable signal EN2 and two unused ports, identified as COORD0 and COOR3. Finally, only the left logical group is bypassed during the bonding1 package option. If logical groups are bypassed, they must be provided in the pinorder file.

If needed, additional unused ports and bypassed logical groups can be added by clicking the  icon in those option fields.

For this example, the second BondingConfiguration is named bonding2 and has unused ports of D1[0], D1[1], D1[2] and D1[3].

Figure 3-6. Configuring Bonding Options in DFTVisualizer



5. Use `report_config_data` to list the current configuration:

```
ANALYSIS>report_config_data $spec
```

6. Once the configuration is reported, you can cut and paste this information into the dofile and read the DFTVisualizer edits using the `read_config_data` command as shown in the example below. Then, for subsequent runs, the configuration edits will already be present in the dofile, making the process repeatable via scripts.

```
read_config_data -in $spec/BoundaryScan -from_string {
BondingConfigurations +{
    BondingConfiguration(default) {
    }
    BondingConfiguration(bonding1) {
        enable_signal : EN2;
        unused_ports : COORD0, COORD3;
        bypassed_logical_groups : left;
    }
    BondingConfiguration(bonding2) {
        unused_ports : D1[2] D1[1] D1[0] D1[3];
    }
}
}
```

7. If you are running Tessent Shell in the dft context with `-no_rtl` and the library does not have a `clock_gating_and` entry, the following operation needs to be completed to create the RTL for the AND clock gater.

```
set_config_value /DftSpecification(car,gate)/use_rtl_cells On
```

8. Complete any other DftSpecification configurations that may be required and continue the rest of the design flow.

Custom Boundary Scan Cells

Custom boundary scan cells can be combined with pad cells in the overall boundary scan chain. Currently, Tessent Shell does not support insertion of custom boundary scan cells, but this functionality will be supported soon. For now, custom boundary scan cells need to already be inserted into the design and connected to the pad cell.

The custom boundary scan cell is described using the same format as found in the `*.tcd_bscan` file. This file is normally created by the `process_dft_specification` when inserting a boundary scan chain into a sub or physical block, and the block contains a `Core(<module_name>)/BoundaryScan` wrapper. For this application, the wrapper will be manually created to specify the custom boundary scan cell and interface. Tessent Shell automatically recognizes this description when matching modules within higher level parent modules and uses it to stitch the boundary scan chain in the parent module.

Example 1

This example shows a sample `.tcd_bscan` file for custom input cells, named `ipad_bscan_combo.tcd_bscan`, and the process used for stitching them with pad cells that are automatically inserted.

The contents of the custom `ipad_bscan_combo.tcd_bscan` file is listed below:

```
Core(ipad_bscan_combo) {
  BoundaryScan {
    Interface {
      select_jtag_input : SJI_in;
      capture_shift_clock : bscan_clk;
      update_clock : update_bscan;
      shift_en : shift_bscan;
      scan_in : bscanIn;
      scan_out : bscanOut;
      scan_out_launch_edge : negedge;
    }
    ExternalPort(PAD) {
    }
    Cell(cell0) {
      function : input;
      bsd1_cell_type : BC_2;
      external_port : PAD;
    }
  }
}
```

The `ipad_bscan_combo.tcd_bscan` file is located in the netlist directory along with the `ipad_bscan_combo.v` verilog file in this example.

```
SETUP>set_context dft -no_rtl

##Read the Tessent library for standard cells and pad cells.
SETUP>read_cell_library ../library/adk_complete.tcelllib

#Read the verilog
SETUP>read_verilog netlist/cpu_top.v
SETUP>set_design_sources -format verilog -Y netlist -extensions v
SETUP>read_verilog netlist/ipad_bscan_combo.v
SETUP>set_current_design cpu_top

##set_dft_specification_requirements cannot be specified until the
design has been read in.
ANALYSIS>set_dft_specification_requirements -boundary_scan On

##Need to set the design level before running check_design_rules
ANALYSIS>set_design_level chip
```

The remainder of the design flow steps are unchanged beyond this point, and would merge at the [Add Constraints](#) step within [Specify and Verify DFT Requirements](#).

Example 2

This example shows a sample .tcd_bscan file for custom output cells. In this case, the netlist will already have the output bscan cells inserted and connected, as well as a combinational cell for the enable. The process used for stitching the custom boundary scan outputs with pad cells that are automatically inserted is the same as that given in [Example 1](#).

```
Core(opad_bscan_en_combo){
  BoundaryScan {
    Interface {
      select_jtag_output : SJO_in;
      force_disable      : forcedis;
      capture_shift_clock : bscan_clk;
      update_clock       : update_bscan;
      shift_en           : shift_bscan;
      scan_in            : bscanIn;
      scan_out           : bscanOut;
      scan_out_launch_edge : negedge;
    }
    ExternalPort(PAD) {
      buffer_type : three_state;
      control_cell : cell1;
    }
    Cell(cell0) {
      function : output;
      bsd1_cell_type : BC_2;
      external_port : PAD;
    }
    Cell(cell1) {
      function : control;
      bsd1_cell_type : BC_2;
      control_enable_value : 1;
    }
  }
}
```

```
}
```

Debugging Failing JtagBscan Simulations

Tessent Shell provides the capability of running and checking testbench simulations. Failing simulation patterns can be identified, viewed and analyzed. One method of accomplishing this will be outlined in this section.

The command for running a set of simulation test benches in Tessent Shell is [run_testbench_simulations](#). This command is normally run with no arguments since it automatically uses the design name, design id and pattern id found in the previously processed [PatternsSpecification](#)(design_name, design_id, pattern_id) wrapper.

The command [check_testbench_simulations](#) is used to check the status of simulations that were previously launched by [run_testbench_simulations](#). Arguments can be passed to generate a status report or a status Tcl dictionary. If no argument is provided, the command will update a status line each second while running and report an error message for any failed simulations when completed.

Step 1

The following example runs a set of testbench simulations based on the design name, design id and pattern id from the previously processed [PatternsSpecification](#). Test bench simulations are checked and any failing pattern simulations are reported.

```
SETUP>run_testbench_simulation
SETUP>check_testbench_simulations
// Error: 1 out of 2 simulations failed:
// JtagBscanPatterns with 943 unexpected miscompares
```

Step 2

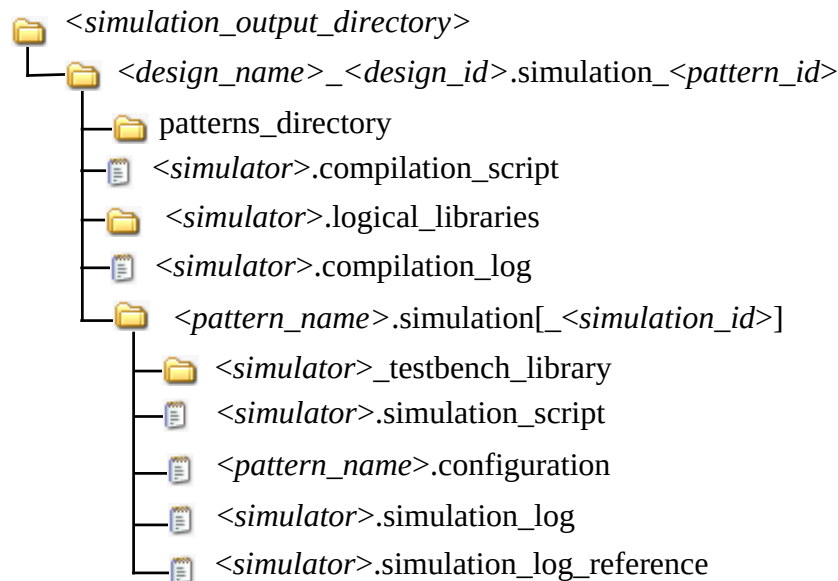
This example shows another way to report testbench simulation status and the resulting output based on the same [run_testbench_simulation](#) of Step 1.

```
SETUP>check_testbench_simulations -report_status
// Simulation status for
// ./simulation_outdir/cpu_top_gate.simulation_signoff
// =====
// -----
// Pattern Name      Status  Unexpected  Missing  Date
//                   Miscompares Miscompares
// -----
// ICLNetwork        pass    0           0        Thu May 07
// JtagBscanPatterns fail    943         0        Thu May 07
```

The `-report_status` argument creates a report listing the simulation status of each pattern found inside the `<design_name>_<design_id>.simulation_<pattern_id>` directory as shown in

Figure 3-7. This report identifies any pattern id that failed testbench simulation and needs to be re-run to capture and store waveform data for further debug in the waveform viewer.

Figure 3-7. Simulation Output Directory Contents



Step 3

The following example shows how you would run simulation on a specific pattern_id that failed testbench simulation and then save the simulation waveforms for viewing in the waveform window for further analysis.

Availability of the failing pattern and design environment should be confirmed since a different PatternsSpecification could have been processed after the patterns that failed were run. The run_testbench_simulations command with the -report_list argument lists the patterns available in the <design_name>_<design_id>.patterns_<pattern_id>/simulation.data_dictionary file. This file is always updated upon successful validation and processing of the PatternsSpecification.

```
SETUP>run_testbench_simulations -report_list
List of pattern(s) for directory
'./tsdb_outdir/patterns/cpu_top_gate.patterns_signoff':
ICLNetwork  JtagBscanPatterns
```

Once the pattern availability is confirmed, as seen in the sample above with the listing of JtagBscanPatterns, the simulations can be re-run with waveform storage enabled.

```
SETUP>run_testbench_simulations -select JtagBscanPatterns \
-store_simulation_waveforms on
```

Running `run_testbench_simulations` with the `-store_simulation_waveforms` argument enabled creates a `vsim.wlf` file in the directory where the `JtagBscanPatterns` were simulated.

In another unix shell window, navigate to the folder where the `vsim.wlf` file was created and open it using the viewer in Questa Modelsim as shown in the example below:

```
UNIX>cd \  
simulation_outdir/cpu_top_gate.simulation_signoff/ \  
JtagBscanPatterns.simulation  
UNIX>vsim -view -vsim.wlf
```

While analyzing the waveforms of the failing patterns, you have the option of rerunning the simulation in the tessent shell window with new files or other options such as `delay_mode_zero`, `unit_delay` or others. After any desired adjustments are made, you can reload the new `vsim.wlf` file in the waveform window to view the results.

Step 4 (Optional)

If for some reason you quit a Tessent -shell session similar to that outlined in [Step 3](#) and returned later to continue analysis, you can read the simulation library in the new Tessent -shell session using the [set_simulation_library_sources](#) command. The session can be restored by following the sequence shown:

```
UNIX>tessent -shell -log simulation_debug.log  
SETUP>set_context patterns  
SETUP>set_simulation_library_sources \  
-v ../library/adk_complete.v -v ../library/picdram.v  
SETUP>run_testbench_simulations -report_list  
List of pattern(s) for directory  
'./tsdb_outdir/patterns/cpu_top_gate.patterns_signoff':  
ICLNetwork JtagBscanPatterns  
  
SETUP>run_testbench_simulations -select JtagBscanPatterns \  
-store_simulation_waveforms on -wait
```

In another unix shell window, navigate to the folder where the `vsim.wlf` file was created and open it using the viewer in Questa Modelsim as shown in the example below:

```
UNIX>cd \  
simulation_outdir/cpu_top_gate.simulation_signoff/ \  
JtagBscanPatterns.simulation  
UNIX>vsim -view -vsim.wlf
```

Related Topics

[run_testbench_simulations](#)

[check_testbench_simulations](#)

[set_simulation_library_sources](#)

Chapter 4

MemoryBIST Insertion with BoundaryScan

This chapter describes the design flow that can be followed if TAP, BoundaryScan and MemoryBIST are to be inserted for a design.

Overview	65
TAP, BoundaryScan and MemoryBIST	66
MemoryBIST Insertion Before Tap and BSCAN	68

Overview

Two example design flows are described that will show different methods of inserting MemoryBIST with BoundaryScan and TAP into a design.

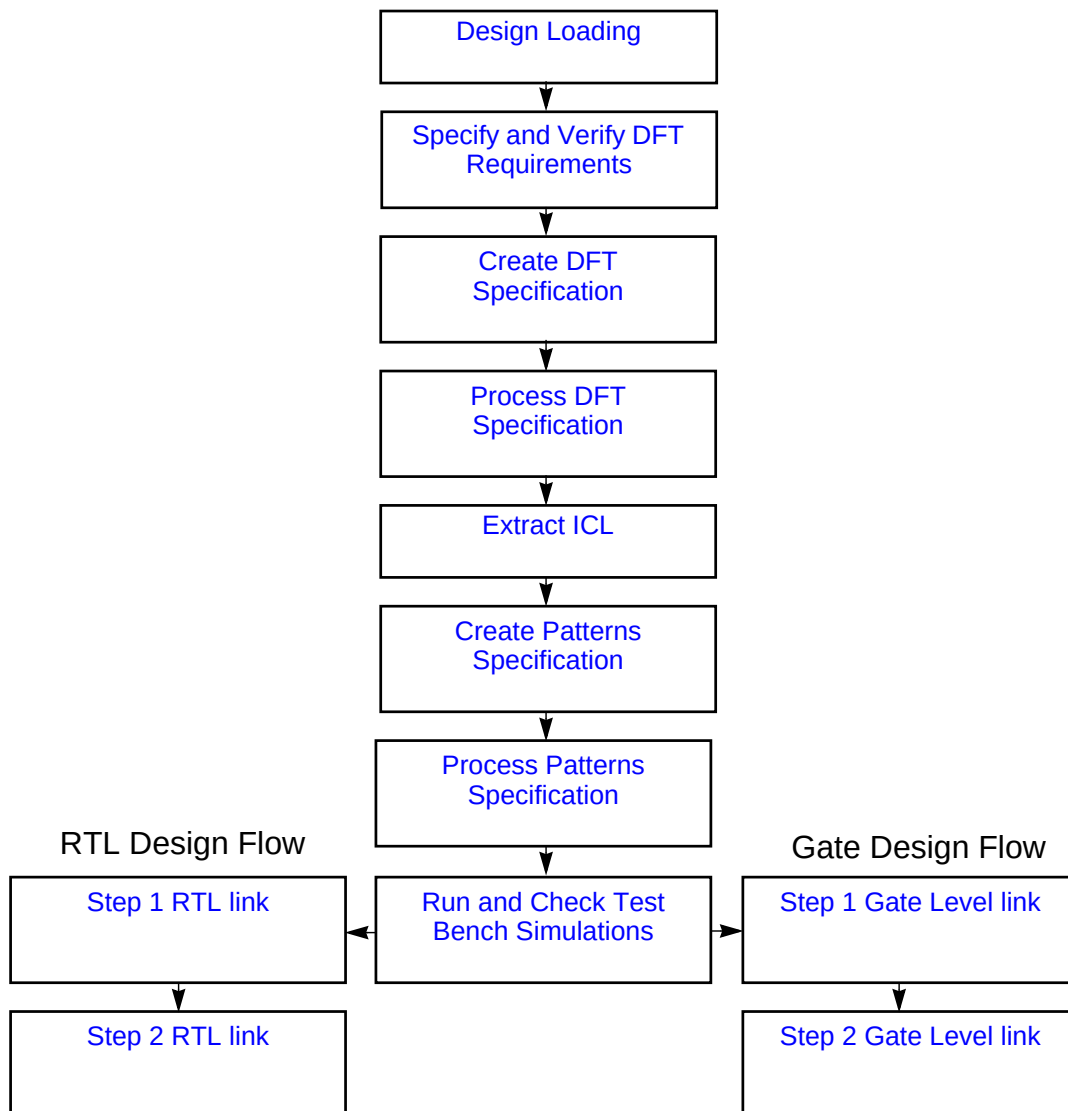
The first example will cover inserting MemoryBIST, BoundaryScan and TAP in a single pass at the chip design level.

The second example will cover inserting MemoryBIST in a first pass within a sub_block or physical_region, then inserting BoundaryScan and TAP in a second pass at the chip level. When memoryBIST needs to be inserted inside a physical_region or a sub_block, then the design level needs to also be set to physical_region or a sub_block. A physical_region describes a design module where physical layout will be completed, whereas a sub_block describes a design module that can be instantiated inside another layout region. The layout region can be another design module or the entire chip.

Further information on MemoryBIST insertion can be found in the [Tessent MemoryBIST User's Manual For Use with Tessent Shell](#).

The design flow that is used to insert MemoryBIST with BoundaryScan and TAP is the same as described in [Getting Started](#) and is shown in [Figure 4-1](#) below.

Figure 4-1. Design Flow for Tessent Shell



TAP, BoundaryScan and MemoryBIST

The example provided in this section covers the design implementation for MemoryBIST, BoundaryScan and TAP all located at the chip design level. The example does this in a single implementation pass where the TAP, BoundaryScan and MemoryBIST are all generated and inserted at the same time. The generation and insertion could also be done for each in three separate passes if desired.

Procedure

1. **Design Loading** steps. In this step, the design and libraries are loaded in:

```
set_context dft -no_rtl
read_cell_library ../library/adk_complete.tcelllib
read_cell_library ../library/memory.lib
set_design_sources -format tcd_memory -V ../library/picdram.memlib
read_verilog ../netlist/cpu_top.v
set_current_design cpu_top
```

2. **Specify and Verify DFT Requirements** steps. In the example below, `set_dft_specification_requirements` has both `-boundary_scan On` and `-memory_test On`. The command `set_design_level` is set to “chip”. The DRC is run when the command `check_design_rules` is issued.

```
set_design_level chip
set_dft_specification_requirements -memory_test on \
                                  -boundary_scan on

set_attribute_value tck_p -name function -value tck
set_attribute_value tdi_p -name function -value tdi
set_attribute_value tms_p -name function -value tms
set_attribute_value trst_p -name function -value trst
set_attribute_value tdo_p -name function -value tdo
set_attribute_value vdd* -name function -value power
set_attribute_value vss* -name function -value ground
add_clocks 0 ramclk_p -Period 5ns
set_boundary_scan_port_options ramclk_p -cell_options clock
check_design_rules
```

3. **Create DFT Specification** steps:

```
set_spec [create_dft_specification]
report_config_data $spec
//display_spec
set_config_value /DftSpecification(cpu_top,gate)/use_rtl_cells On
```

4. **Process DFT Specification and Extract ICL** steps:

```
process_dft_specification
extract_icl
```

5. **Create Patterns Specification** steps:

```
set_pat_spec [create_pat_specification]
report_config_data $pat_spec
```

6. **Process Patterns Specification** step:

```
process_patterns_specification
```

7. **Run and Check Test Bench Simulations** steps:

```
set_simulation_library_sources -v ../library/adk_complete.v \
                               -v ../library/picdram.v
run_testbench_simulations
```

```
check_testbench_simulations
check_testbench_simulations -report_status
```

Related Topics

[set_simulation_library_sources](#)

[run_testbench_simulations](#)

[check_testbench_simulations](#)

MemoryBIST Insertion Before Tap and BSCAN

The example provided in this section covers the initial design insertion for MemoryBIST on a sub_block or physical_region, and finally the insertion of TAP and BoundaryScan at the chip level on a second pass.

If default tsdb_outdir is not used for MemoryBIST, then while inserting TAP and BoundaryScan, use the open_tsdb command to point to the tsdb outdir of where the MemoryBIST is inserted.

Procedure

1. MemoryBIST insertion: [Design Loading](#) steps:

```
set_context dft -rtl
read_cell_library ../library/adk.tcelllib
set_design_sources -format verilog -y {../data/design/mem
../data/design/rtl} -extension v
read_verilog ../data/design/rtl/blockA.v
set_current_design blockA
report_memory_instances
```

2. MemoryBIST insertion: [Specify and Verify DFT Requirements](#) steps. MemoryBIST is inserted on a sub_block level.

```
set_design_level sub_block
set_dft_specification_requirements -memory_test on
add_clock CLK -period 12ns -label clka
check_design_rules
```

3. MemoryBIST insertion: [Create DFT Specification](#) steps:

```
create_dft_specification
report_config_data
report_config_syntax DftSpecification/MemoryBist
```

4. MemoryBIST insertion: [Process DFT Specification](#) and [Extract ICL](#) steps:

```
process_dft_specification
extract_icl
```

5. MemoryBIST insertion: [Create Patterns Specification](#) steps:

```
set spec [create_pattern_spec]
report_config_data $spec
```

6. MemoryBIST insertion: [Process Patterns Specification](#) step:

```
process_patterns_specification
```

7. MemoryBIST insertion: Run & Check Test Bench Simulations steps:

```
run_testbench_simulations
check_testbench_simulations
```

The prior seven steps need to be repeated for any other sub_blocks or physical_regions with memories that need MemoryBIST insertion before the design level is changed to chip level.

At the chip-level, if you have memories that need MemoryBIST insertion, you will create a DFT Specification as shown below which inserts the MemoryBIST as well as the TAP and BoundaryScan cells.

Note



If the default “tsdb_outdir” is not used for MemoryBIST insertion, then you must use the [open_tsdb](#) command to point to the “tsdb_outdir” for where the MemoryBIST is inserted while inserting TAP and BoundaryScan cells.

1. BoundaryScan and Tap insertion: [Design Loading](#) steps:

```
set_context dft -rtl
read_cell_library ../library/adk.tcelllib
read_verilog ../data/design/rtl/top.v \
    ../data/design/fusebox/*.v

set_current_design top
```

2. BoundaryScan and Tap insertion: [Specify and Verify DFT Requirements](#) steps. The TAP signals can be specified here, as shown in Step 2 of [TAP, BoundaryScan and MemoryBIST](#), or the specified by the DefaultsSpecification as assumed in this example procedure.

```
set_dft_specification_requirements -boundary_scan on
set_design_level chip
add_clocks clka -period 3ns
add_clocks clk_b -period 12ns
set_attribute_value vddq -name function -value power
set_attribute_value vss -name function -value ground
check_design_rules
```

3. BoundaryScan and Tap insertion: [Create DFT Specification](#) steps:

```
set spec [create_dft_spec]
cat top.bisr_segment_order
report_conf_data $spec
```

4. BoundaryScan and Tap insertion: [Process DFT Specification](#) and [Extract ICL](#) steps:

```
process_dft_specification  
extract_icl
```

5. BoundaryScan and Tap insertion: [Create Patterns Specification](#) steps:

```
set_defaults_value \  
  PatternsSpecification/SignOffOptions/ \  
  simulate_instruments_in_lower_physical_instances on  
set spec [create_patterns_specification]
```

6. BoundaryScan and Tap insertion: [Process Patterns Specification](#) step:

```
process_patterns_specification
```

7. BoundaryScan and Tap insertion: Run and Check Test Bench Simulations steps:

```
set_simulation_library_sources -v \  
  ../library/verilog/adk.v  
run_testbench_simulations  
exit
```

Related Topics

[run_testbench_simulations](#)

[set_simulation_library_sources](#)

Chapter 5

Tap, BoundaryScan and LPCT Type 2 TestKompress

This chapter describes the steps that are required to use the TAP to control TestKompress, where the TAP's TDI and TDO pins are used as the EDT channel in and channel out pins. The BoundaryScan chain will be segmented into smaller Reduced Pin Count Test (RPCT) segments so they can also be part of the logic testing.

Overview	71
Design Flow	72

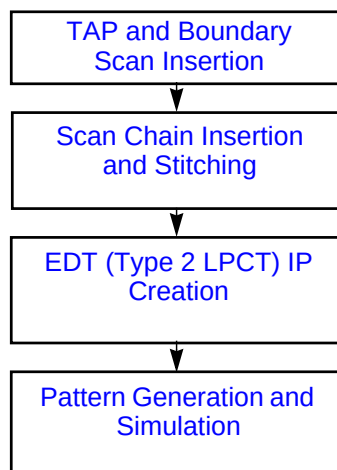
Overview

The benefit of implementing TestKompress with a Tap and Type 2 Low Pin Count Test (LPCT) controller and segmenting boundary scan cells into RPCT segments is that the combinational logic that is present between the boundary scan cell and the first tier or level of functional flops is tested during logic testing with TestKompress.

The procedure outlined in this chapter will provide an example design implementing this architecture to serve as a guide.

The design flow shown in [Figure 5-1](#) shows the high-level overview of the steps that will be implemented.

Figure 5-1. Tap, Boundary Scan and LPCT Type 2 TK Design Flow



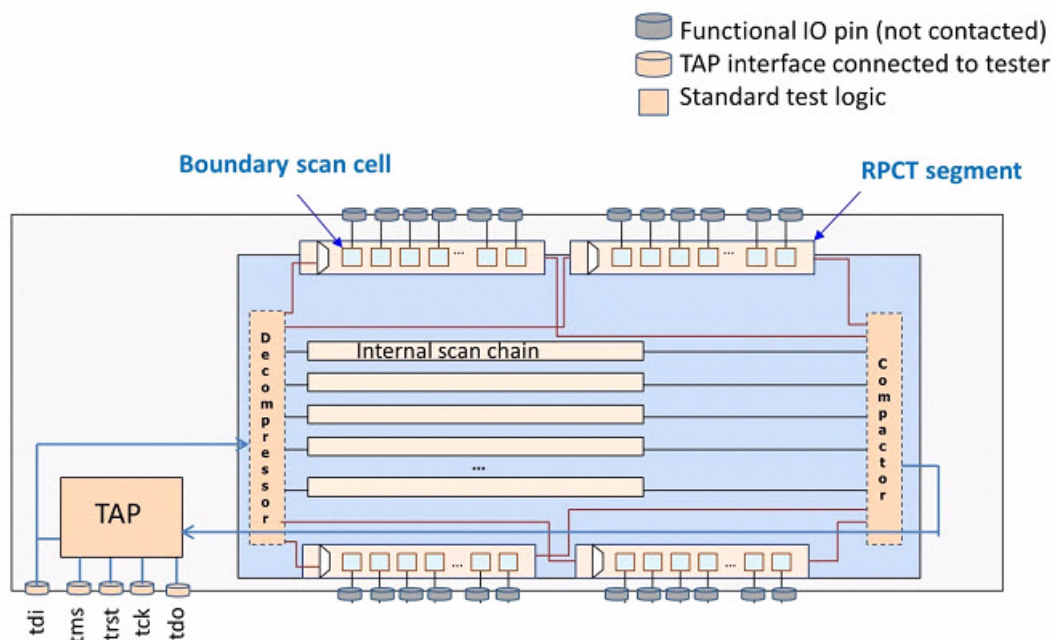
The TAP and boundary scan chains will be inserted first, followed by scan insertion and stitching of the functional design flops. The segmented RPCT boundary scan chains are declared as pre-existing scan chains for the scan insertion step.

In the IP creation step, the signals from the TAP controller that are needed to be connected to the Type 2 LPCT controller are specified. The TAP controller's TDI and TDO pins will be used as the EDT channel in and channel out pins respectively.

Finally, patterns are created with TDI and TDO as the single channel for TestKompress. The TAP states are then cycled and validated via verilog simulation to ensure the design's integrity.

Figure 5-2 shows how a single boundary scan chain that is connected between the TDI and TDO of the TAP controller gets divided into smaller RPCT (Reduced Pin Count Test) segments and how it is connected between the decompressor and the compactor of the TestKompress IP. For further information, refer to “[Type 2 LPCT Controller](#)” in the *Tessent TestKompress User's Manual*.

Figure 5-2. Tap, Boundary Scan and Type 2 LPCT TestKompress Implementation



Design Flow

The design steps, considerations and example scripts to implement this architecture are described in the following sections.

TAP and Boundary Scan Insertion

The first step is to insert the TAP and Boundary scan chain.

Procedure

- The [DFT Flow Using Tessent Shell](#) outlined in [Getting Started](#) is followed to insert the TAP and Boundary Scan. The example script shown below provides more specific detail and can also be used as a guide.
- Estimate the scan chain length for the functional design cells and apply this length to segment the boundary scan into smaller segments.
 - i. If the scan chain length for the functional design cells is not known, pick a number for the boundary scan chain length to ensure it will not be the longest chain and impact compression.
 - ii. Use `max_segment_length_for_logictest` property in the [BoundaryScan](#) wrapper to specify how many boundary scan cells are to be in each scan chain.

Examples

This example script inserts a TAP and boundary scan chain following the [DFT Flow Using Tessent Shell](#) steps. A TDR is also connected to the TAP controller that provides control for the boundary scan chain to be either a single chain, or segmented and used with logic testing. The use of `max_segment_length_for_logictest` property is also shown and sets the boundary scan chain length to 250 cells.

If you need to perform custom editing between the insertion of DFT components and the saving of the design, you can define a Tcl proc with the name “`process_dft_specification.post_insertion`”. This proc will automatically be called during `process_dft_specification` after all DFT components have been inserted and before the `write_design` command is invoked. The example script calls makes use of this technique to insert clock control logic that provides the proper clocking for segmented boundary scan chains during logic testing. The contents of this file are provided in [Figure 5-3](#) and can be customized to meet your design requirements. For more information on this method, refer to the [process_dft_specification](#) command description.

```
##Design Loading
set_context dft -no_rtl
read_cell_library ../../libs/adk_complete.tcelllib
read_cell_library ../../libs/memory.lib
read_verilog ../netlist/cpu_top.v
set_current_design cpu_top

##Specify and Verify DFT Requirements
set_dft_specification_requirements -boundary_scan On
set_design_level chip
set_attribute_value tck_p -name function -value tck
set_attribute_value tdi_p -name function -value tdi
```

```
set_attribute_value tms_p -name function -value tms
set_attribute_value trst_p -name function -value trst
set_attribute_value tdo_p -name function -value tdo
set_boundary_scan_port_options ramclk_p -cell_options clock
check_design_rules

##Create DFTSpecification
set_spec [create_dft_specification]
set_config_value
$spec/BoundaryScan/max_segment_length_for_logic_test 250
read_config_data -in $spec/IjtagNetwork/HostScanInterface(tap)/ \
    Tap(main)/HostIjtag(1) -from_string {
        Tdr(logic_enable) {
            DataOutPorts +{
                count      : 3;
                port_naming : ltest_enable, bypass_enable, \
                            low_power_enable;
            }
        }
    }
report_config_data $spec
set_config_value /DftSpecification(cpu_top,gate)/use_rtl_cells On

##Custom proc to be run after process_dft_specification
source post_dft_insertion_procedure.tcl

##Process DFTSpecification
process_dft_specification

##Inspect the results to confirm the boundary scan is now segmented
##into smaller chain segments, as described in Step 5 of
## Dividing Boundary Scan for Logic Test. The maximum length was
##set by max_segment_length_for_logic_test to 250 flops.
get_instrument_dictionary -list
format_dictionary [get_instrument_dictionary
mentor::jtag_bscan::DftSpecification logic_test_scan_chains]

##Save the above into a separate file called
##logic_test_scan_chains.dictionary for later reference.
set fp [open logic_test_scan_chains.dictionary w]
puts $fp "set logic_test_scan_chains {"
puts $fp [format_dictionary [get_instrument_dictionary
mentor::jtag_bscan::DftSpecification logic_test_scan_chains ] ]
puts $fp "}"
close $fp

##Extract ICL
extract_icl

##Create Patterns Specification
set_pat_spec [create_pat_specification]

##Without this the select of the mux on the functional clocks will
##be X, causing simulation failures
set_config_value
$pat_spec/AdvancedOptions/ConstantPortSettings/scan_en1 0
##Process Patterns Specification
```

```
process_patterns_specification

##Run & Check Test Bench Simulation
set_simulation_library_sources -v ../../libs/adk_complete.v \
    -v ../../libs/pads.v -v ../../libs/ram.v
run_testbench_simulations
check_testbench_simulations
```

**Figure 5-3. post_dft_insertion_procedure.tcl Example File
(design name = cpu_top)**

```
proc process_dft_specification.post_insertion {cpu_top args} {

##The following is needed for controlling the clocks for segmented
##boundary scan chain during logic/scan test

##Creating a scan_en1 port
create_port scan_en1
##Deleting the existing connection on logic_test_enable port
delete_connection
cpu_top_gate_tessent_bscan_logical_group_DEF_inst/logic_test_enable
##Creating connecting from the TDR to control logic_test_enable
##When this TDR is set to 1, then in logic testing mode
create_connection
cpu_top_gate_tessent_tdr_logic_enable_inst/ltest_enable
cpu_top_gate_tessent_bscan_logical_group_DEF_inst/logic_test_enable

##The clockbscan, updatebscan and shiftBscan2Edge are intercepted by
##a mux. The other inputs of the clockbscan, updatebscan is tck
##whereas the other input of shiftBscan2Edge is scan_en1.
set tck_tap_point \
    [get_fanins cpu_top_gate_tessent_bscan_interface_I/tck]
intercept_connection cpu_top_gate_tessent_bscan_interface_I/ \
    to_bscan_capture_shift_clock \
        -cell_function_name mux \
        -input2 $tck_tap_point \
        -select cpu_top_gate_tessent_bscan_logical_group_DEF_inst/ \
            logic_test_enable \
        -leaf_instance_prefix clockBscan_
intercept_connection cpu_top_gate_tessent_bscan_interface_I/ \
    to_bscan_update_clock \
        -cell_function_name mux \
        -input2 $tck_tap_point \
        -select cpu_top_gate_tessent_bscan_logical_group_DEF_inst/ \
            logic_test_enable \
        -leaf_instance_prefix updateBscan_
intercept_connection cpu_top_gate_tessent_bscan_interface_I/ \
    to_bscan_shift_en \
        -cell_function_name mux \
        -input2 scan_en1 \
        -select cpu_top_gate_tessent_bscan_logical_group_DEF_inst/ \
            logic_test_enable \
        -leaf_instance_prefix shiftBscan2Edge_
```

```
##Do not need muxes to choose between functional clock and tck when
##OCC is present.
##Adding muxes to all the functional clocks in the design
intercept_connection /inpad0/C -cell_function_name mux \
    -input2 $tck_tap_point -select scan_en1 \
    -leaf_instance_prefix clk1_p_
intercept_connection /inpad1/C -cell_function_name mux \
    -input2 $tck_tap_point -select scan_en1 \
    -leaf_instance_prefix clk2_p_
intercept_connection /inpad2/C -cell_function_name mux \
    -input2 $tck_tap_point -select scan_en1 \
    -leaf_instance_prefix clk3_p_
intercept_connection /inpad3/C -cell_function_name mux \
    -input2 $tck_tap_point -select scan_en1 \
    -leaf_instance_prefix clk4_p_
intercept_connection /inpad4/C -cell_function_name mux \
    -input2 $tck_tap_point -select scan_en1 \
    -leaf_instance_prefix ramclk_p_
```

Related Topics

Scan Chain Insertion and Stitching

The second step is to insert scan chains for the rest of the functional design logic.

Prerequisites

- Complete [TAP and Boundary Scan Insertion](#) steps.

Procedure

Use the steps outlined in the example script provided below as a guide to complete this stage of the design flow. Supporting scripts are provided in the Figures following the example.

In this stage of the design flow, the boundary scan chains are declared as pre-existing scan chains using the [add_scan_chains](#) -internal property. The icl file for the design created in [TAP and Boundary Scan Insertion](#) will be read in using [read_icl](#), and synthesized along with the functional design scan chains. A PDL file is also created and read in for the TAP controller and the TDR that is used to enable a segmented boundary scan chain.

Examples

```
set_context dft -scan
read_cell_library ../library/adk_complete.tcelllib
read_verilog ../from_step1/cpu_top.v_full
read_verilog ../from_step1/synthesized/boundary_scan_cells.v
read_verilog ../from_step1/synthesized/tessent_tap_main.v
```

```
##Reading icls and pdls before set_current_design
read_icl ../from_step1/tsdb_outdir/dft_inserted_designs/ \
    cpu_top_gate.dft_inserted_design/cpu_top.icl
dofile ../dofiles/cpu_top_gate_tessent_tap_main.pdl
dofile ../dofiles/cpu_top_gate_tessent_tdr_logic_enable.pdl

set_current_design cpu_top

## Add clock definitions
add_clocks 0 tck_p
add_clocks 0 clk1_p
add_clocks 0 clk2_p
add_clocks 0 clk3_p
add_clocks 0 clk4_p
add_clocks 0 ramclk_p

add_input_constraints trst_p -C1
add_input_constraints tms_p -C0

##Scan Enable scan_en1 as pre-Existing
set_scan_enable scan_en1
add_input_constraints scan_en1 -C0
add_nonscan_instances cpu_top_gate_tessent_tap_main_inst
add_nonscan_instances cpu_top_gate_tessent_tdr_logic_enable_inst

## Reading in pre-existing bscan chains
dofile ../dofiles/e_chains.dofile

set_system_mode analysis
insert_test_logic -max_length 250 -clock merge \
    -edge merge -new_scan_po
report_test_logic
report_scan_chains
write_design -output generated/cpu_top_scan.v -replace
write_atpg_setup generated/cpu_scan -replace
```

Figure 5-4. Example cpu_top_gate_tessent_tap_main.pdl File

```
## Called from the Test_Setup procedure in Figure 5-7.
iProcsForModule cpu_top_gate_tessent_tap_main
iProc tap_main_setup { } {
    //SelectJtagOutput is 0 so the value from the core is captured into
    //the output boundary scan cells.
    iWrite select_jtag_output 0b0
    //SelectJtagInput is 1 so the value from the boundary scan register
    //or cell is captured into the first level of functional flops.
    iWrite select_jtag_input 0b1
iApply
}
```

Figure 5-5. Example cpu_top_gate_tessent_tdr_logic_enable.pdl File

```
## Called from the Test_Setup procedure in Figure 5-7.
iProcsForModule cpu_top_gate_tessent_tdr_logic_enable
```

```
iProc logic_enable { } {  
    iWrite tdr[2:0] 0b100  
iApply  
}
```

Figure 5-6. Example e_chains.dofile

```
add_scan_groups group1 existing_chains.testproc  
add_scan_chains -internal chain0 group1 \  
    cpu_top_gate_tessent_bscan_logical_group_DEF_inst/\  
    clk1_p_logic_scanin \  
    cpu_top_gate_tessent_bscan_logical_group_DEF_inst/\  
    CELL249_BSCAN_SO  
add_scan_chains -internal chain1 group1 \  
    cpu_top_gate_tessent_bscan_logical_group_DEF_inst/\  
    expdout_p_6_logic_scanin \  
    cpu_top_gate_tessent_bscan_logical_group_DEF_inst/\  
    CELL0_BSCAN_SO
```

Figure 5-7. Example existing_chains.testproc File

```
set time scale 1ns;  
alias int_clocks = clk1_p, clk2_p, clk3_p, clk4_p;  
timeplate global =  
    force_pi 0;  
    measure_po 15;  
    pulse tck_p 25 50; // tck_p  
    pulse clk1_p 25 50; //  
    pulse clk2_p 25 50; //  
    pulse clk3_p 25 50; //  
    pulse clk4_p 25 50; //  
    period 100;  
end; // timeplate global  
  
procedure Test_Setup = // Describes the setup phase to be applied  
                        //once at the beginning of test  
timeplate global;  
iCall cpu_top_gate_tessent_tap_main_inst.tap_main_setup ;  
iCall cpu_top_gate_tessent_tdr_logic_enable_inst.logic_enable ;  
end; // procedure Test_Setup }  
  
procedure Shift = // Describes one clock cycle of shift.  
timeplate global;  
cycle =  
    force scan_en1      1;  
    force trst_p        1; // trst_p  
    pulse tck_p;         // tck_p  
    force tms_p         0; // tms_p  
    force_sci;  
    measure_sco;  
end;  
end; // procedure Shift
```

```

procedure Load_Unload =//Proceedure to load/unload the scan chains.
timeplate global;
  cycle =
    force trst_p      1; // trst_p
    force tck_p       0; // tck_p
    force tdi_p       0; // tdi_p
    force int_clocks   0;
    force tms_p       0; // tms_p
end;
  apply Shift 250; //Auto adjusted by Fastscan to match the longest
                  //scan chain
end; // procedure Load_Unload

```

EDT (Type 2 LPCT) IP Creation

The third step is to create the Embedded Deterministic Testing (EDT) compression logic IP .

Prerequisites

- Complete [TAP and Boundary Scan Insertion](#) steps.
- Complete [Scan Chain Insertion and Stitching](#) steps.

Procedure

1. Use the steps outlined in the example script provided below as a guide to complete this stage of the design flow. The example shows the settings needed for LPCT Type 2 IP creation.

Note



The test_logic_reset signal from the Tessent TAP needs to be specified as active low since it is driven as active low. Failure to do this will result in failing simulations and a condition that is difficult to debug.

2. Synthesize the EDT IP logic RTL that was generated into core netlist Verilog gates using the Synopsis Design Compiler script generated during the IP creation. For more information, see [“Inserting EDT Logic During Synthesis”](#) in the *Tessent TestKompress User’s Manual*.

Examples

```

##Using a Type 2 LPCT Controller
set_lpct_controller On -TAp_controller_interface On \
  -Generate_scan_enable On
set_lpct_controller -shift_control clock

##LPCT Pin connections from LPCT controller
set_lpct_pins output_scan_en scan_en1

```

```
set_lpct_pins TEST_Clock_connection \  
    clk1_p_mux/A1 clk2_p_mux/A1 clk3_p_mux/A1 clk4_p_mux/A1 \  
    clockBscan_mux/A1 updateBscan_mux/A1  
  
##Use the TAP's TDI and TDO as the external channel to drive EDT  
set_edt_options -location internal -channels 1  
set_edt_pins input_channel 1 tdi_p tdi_i/C  
set_edt_pins output_channel 1 tdo_p tdo_i/I  
  
##LPCT Pin connections to LPCT controller pins  
set_lpct_pins reset - cpu_top_gate_tessent_tap_main_inst/\  
    test_logic_reset -active low  
set_lpct_pins capture_dr - cpu_top_gate_tessent_tap_main_inst/\  
    capture_dr_en  
set_lpct_pins shift_dr - cpu_top_gate_tessent_tap_main_inst/\  
    shift_dr_en  
set_lpct_pins update_dr - cpu_top_gate_tessent_tap_main_inst/\  
    update_dr_en  
set_lpct_pins test_mode -  
cpu_top_gate_tessent_tdr_logic_enable_inst/\  
    ltest_enable  
set_lpct_pins tms tms_p tms_i/C  
set_lpct_pins clock tck_p tck_i/C
```

Related Topics

[“Low Pin Count Test Controller”](#) in *Tessent TestKompress User’s Manual*

[“Inserting EDT Logic During Synthesis”](#) in *Tessent TestKompress User’s Manual*.

Pattern Generation and Simulation

The final process is generating the compressed test patterns and verifying the integrity of the inserted test circuitry and scan chains through simulation.

This section will present guidelines and tips for successful implementation of this stage for the architecture discussed in this chapter, however some will apply equally well in other applications. For further information on generating and verifying EDT test patterns, refer to [“Generating/Verifying Test Patterns”](#) in the *Tessent TestKompress User’s Manual*.

For the implementation described in this chapter, make sure that during the [EDT \(Type 2 LPCT\) IP Creation](#) phase, the [set_lpct_pins](#) TEST_clock_connection property is used correctly as described in the [Examples](#) for that section. If there are no On-Chip-Clock (OCC) generators in the design, then insert TCK for all the functional clocks. The TCK clock needs to be controlled from the output of the LPCT controller.

Note



Read this next section carefully if you are planning on inserting OCC in your design.

The OCC generator is inserted only for functional clocks. The TCK clock will have a clock gating function added and will not receive an OCC generator.

There are two modes in which patterns can be generated during pattern generation - a SLOW and FAST capture mode. During SLOW capture mode, the slow speed clock is used for both shift and capture during “stuck-at” testing. In this mode (SelectJtagOutput = 0 and SelectJtagInput = 1), the logic states into and out of the core are being sourced and captured by the boundary scan registers. However, during the FAST capture mode (SelectJtagOutput = 1 and SelectJtagInput = 1), the logic state from the core side is not captured in the output boundary scan cells. If not setup correctly, what ATPG predicts happens will not match simulation results, creating errors that are very difficult to diagnose.

In some designs, it may be that the “at-speed” coverage is very low due to the fact that any interaction between the boundary scan cells and the core is not covered. If the boundary scan cells can be synthesized to the same frequency as the fast capture clock, then during transition testing the interaction can be tested “at-speed”. Typically, this is not desired because during boundary scan test the clock used is TCK, which is a slow speed clock.

Appendix A

Tessent Core Description

This section describes the configuration data syntax used to describe the following macro module types: core and boundary scan segments.

This appendix uses the following syntax conventions when documenting wrappers and properties used in the library descriptions.

Table A-1. Conventions for Command Line Syntax

Convention	Example	Usage
UPPercase	-Static	Required argument letters are in uppercase; in most cases, you may omit lowercase letters when entering literal arguments, and you need not enter in uppercase. Arguments are normally case insensitive.
Boldface	set_fault_mode <u>Uncollapsed</u> Collapsed	A boldface font indicates a required argument.
[]	exit [-force]	Square brackets enclose optional arguments. Do not enter the brackets.
<i>Italic</i>	dofile <i>filename</i>	An italic font indicates a user-supplied argument.
{ }	add_ambiguous_paths { <i>path_name</i> -All} [-Max_paths <i>number</i>]	Braces enclose arguments to show grouping. Do not enter the braces.
	add_ambiguous_paths { <i>path_name</i> -All} [-Max_paths <i>number</i>]	The vertical bar indicates an either/or choice between items. Do not include the bar in the command.
Underline	set_dofile_abort <u>ON</u> OFF	An underlined item indicates either the default argument or the default value of an argument.
...	add_clocks <i>off_state</i> <i>primary_input_pin...</i> [-Internal]	An ellipsis follows an argument that may appear more than once. Do not include the ellipsis when entering commands.

Table A-2. Syntax Conventions for Configuration Files

Convention	Example	Usage
<i>Italic</i>	scan_in : <i>port_pin_name</i> ;	An italic font indicates a user-supplied value.
Underline	wgl_type : <u>generic</u> lsi;	An underlined item indicates the default value.
	logic_level : <u>both</u> high low;	The vertical bar separates a list of values from which you must choose one. Do not include the bar in the configuration file.
...	port_naming : <i>port_naming</i> , ...;	Ellipses indicate a repeatable value. The comment “// repeatable” also indicates a repeatable value.
//	// default: ijtag_so	The double slash indicates the text immediately following is a comment and tells the tool to ignore the text.

Core	85
BoundaryScan	86

Core

In Tessent Shell, descriptions of 'core' elements, like the memory library, the boundary scan information, or the fuse box interface, are presented to the tool in form of TCD files (Tessent Core Description files). After loading, they are hierarchically organized under the 'Core' root entry, which is unique for a given module name.

Usage

```
Core(module_name) {  
    Memory {  
    }  
    BoundaryScan {  
    }  
    FuseBoxInterface {  
    }  
}
```

Description

The Core wrapper collects all TCD data read into the tool. Such descriptions are automatically read in during module matching. See the [set_design_sources](#) -format tcd_memory command description for information about where they are looked for. See the [read_core_descriptions](#) command description to learn how to read them in explicitly.

You can also report on the loaded TCD information. You do this using the [report_config_data](#) command. An example is "report_config_data Core(ModuleName)/Memory -partition tcd", which report the contents of the Memory entries under Core. To see the supported syntax, use the [report_config_syntax](#) command, for example "report_config_syntax Core/Memory".

Parameters

- *module_name*

The name of the module, equivalent of the current design module name. You don't need to specify this when loading a memory TCD file. The tool will auto-generate and auto-configure the Core-level wrapper for you.

Related Topics

[set_design_sources](#)

[report_config_data](#)

[set_module_matching_options](#)

[read_core_descriptions](#)

[report_config_syntax](#)

BoundaryScan

Specifies the embedded boundary scan chain already implemented into the design `module_name`.

Usage

```
Core(module_name) {  
  BoundaryScan {  
    Interface {  
    }  
    CustomBsdlCellInfo {  
      cell_type_id : cell_info_description; // repeatable  
    }  
    ExternalPort(port_name) {  
    }  
    Cell(id) {  
    }  
  }  
}
```

Description

Describes the pads and boundary-scan cells contents of a module *module_name* already instantiated in the design. Such descriptions are automatically read in during module matching. See the [set_design_sources](#) -format tcd_bscan command description for information about where they are looked for. See the [read_core_descriptions](#) command description to learn how to read them in explicitly. See the [set_module_matching_options](#) command description for information about the name matching process. Note that the legacy LogicVision .lvbscan format is supported natively and is automatically translated into this format when read.

To see the content of a read-in Core(ModuleName)/BoundaryScan, use the “[report_config_data](#) Core(ModuleName)/BoundaryScan -partition tcd” command.

To see the supported syntax, use the “[report_config_data](#) [get_config_value Core/BoundaryScan -partition meta:tcd -object]” command.

Arguments

None.

Interface

This section describes the common ports of the boundary scan hardware of the module *module_name*.

Usage

```
Core(module_name) {  
  BoundaryScan {  
    Interface {  
      select                : port_name;  
      force_disable         : port_name;  
      select_jtag_input     : port_name;  
      select_jtag_output    : port_name;  
      select_jtag_enable    : port_name;  
      capture_shift_clock   : port_name;  
      capture_shift_clock_inv : port_name;  
      bscan_clock           : port_name;  
      update_clock          : port_name;  
  
      capture_en            : port_name;  
      shift_en              : port_name;  
      update_en             : port_name;  
  
      scan_in               : port_name;  
      scan_out              : port_name;  
      scan_out_launch_edge  : posedge | negedge;  
  
      ac_init_clock0        : port_name;  
      ac_init_clock1        : port_name;  
      ac_signal             : port_name;  
      ac_mode_enable        : port_name;  
    }  
  }  
}
```

Description

This section describes the common ports of the boundary scan hardware of the module *module_name*. These ports usually connect to a higher level boundary scan interface block or to a TAP.

Arguments

- **select** : *port_name*

This signal is used to enable the boundary-scan register logic in the module. If this signal is active the capture, shift and update enable signals effect the boundary scan register. The signal is furthermore used to gate the clocks bscan_clock, capture_shift_clock and capture_shift_clock_inv. The signal is optional and it assumed to be active high.

- **force_disable** : *port_name*

This disables all pad cell drivers. The external ports will show a Z signal. This is limited to those pad cells that can be disabled. The signal overrides the settings from the enable

boundary scan cells. The signal is active high. The signal is needed to support the HIGHZ instruction.

- `select_jtag_input : port_name`

This is the select signal of the SJI multiplexer that switches between the pad cells from `_pad` port and the output of the boundary scan cell. The output of the multiplexer is connected to the core. When the signal is high it selects the boundary scan cell output. This signal is mandatory, in case the module contains output or bidirectional boundary scan cells.

- `select_jtag_output : port_name`

This is the select signal of the SJO multiplexer that switches between the core signal and the output of the boundary scan cell. The output of the mux is connected to the pad cells to `_pad` port. When the signal is high it selects the boundary scan cell output. The signal is mandatory, in case the module contains output or bidirectional boundary scan cells.

- `select_jtag_enable : port_name`

This is the select signal of the SJI multiplexer that switches between the functional pad enable and the output of the enable boundary scan cell. The output of the multiplexer is connected to the pad enable port. When this signal is high it selects the output of the boundary scan cell. This entry is optional. The select of the SJE multiplexer can be connected to `select_jtag_output` instead of this dedicated signal, when this signal is not specified.

- `capture_shift_clock: port_name`

This is a gated version of the test clock. The clock should be active during shift and capture cycles. The inactive state of the clock is the low state. You need to specify either `bscan_clock` or `capture_shift_clock` or `capture_shift_clock_inv`.

- `capture_shift_clock_inv : port_name`

Inverted version of `capture_shift_clock`. This is the legacy clock timing that is used in ETAssemble. You need to specify either `bscan_clock` or `capture_shift_clock` or `capture_shift_clock_inv`.

- `bscan_clock : port_name`

Clock that needs to be continuously running during the boundary scan test. This is usually the test clock or a gated version of the test clock that is disabled outside of the boundary scan test. You need to specify either `bscan_clock` or `capture_shift_clock` or `capture_shift_clock_inv`.

- `update_clock : port_name`

A gated clock that pulses the update register in the boundary scan cell. The disabled state of the clock is the low state. You need to specify this clock when using the `capture_shift_clock` or the `capture_shift_clock_inv` clock. You need to specify either this entry or `update_en`.

- **capture_en** : *port_name*
When this signal is high the boundary scan cells will capture the value from either pad, the core or the update register dependant on the type of the boundary scan cell. You need to specify this signal when you use bscan_clock. The signal is active high.
- **shift_en** : *port_name*
This is the shift enable of the boundary scan register. It is the select of the scan multiplexer inside the boundary scan cells and a high value here will connect the boundary scan cell to a scan chain. This entry is mandatory.
- **update_en** : *port_name*
When this signal is high the update elements in the boundary scan cells take over the value of the boundary scan register. This entry is needed when you use bscan_clock. You need to specify either this entry or update_clock.
- **scan_in** : *port_name*
This is the scan input of the boundary scan segment inside the module. It is used to shift in data from the TAP or a boundary scan interface block or from other parts of the boundary scan register.
- **scan_out** : *port_name*
This is the scan output of the boundary scan segment inside the module. It is used to shift out data to the TAP or a boundary scan interface block or to other parts of the boundary scan register.
- **scan_out_launch_edge** : posedge | negedge
This entry is only valid, if scan_out is defined. This specifies the edge on which the data on the scan output is valid.
- **ac_init_clock0** : *port_name*
This clock is used to trigger the initialization of the test receiver in AC input and AC bi-directional pad cells. The off state of this clock is low.
- **ac_init_clock1** : *port_name*
This clock is used to trigger the initialization of the test receiver in AC input and AC bidirectional pad cells.. The off state of this clock is high.
- **ac_signal** : *port_name*
This signal gives the pulse(s) for the test pulse or train according to the IEEE 1149.6 standard. This signal is mandatory when the module contains AC output or inout cells.
- **ac_mode_enable**: *port_name*
This enables the AC functionality in the described module. This signal enables the input AC functionality and the output functionality for those ports that don't have an AC select cell.

ExternalPort

This section describes the properties of a port that will be connected directly to a top level port and the connected pad cell.

Usage

```
Core(module_name) {  
  BoundaryScan(module_name) {  
    ExternalPort(port_name) {  
  
      differential_inverse_of      : port_name;  
      differential_type            : voltage | current;  
  
      buffer_type                 : three_state | two_state |  
                                  low_only | high_only;  
      highz_during_force_disable : on | off | auto;  
      pull_resistor               : high | low | none;  
  
      control_cell                : cell_id;  
  
      ac_hp_time                  : time;  
      ac_lp_time                  : time;  
      ac_hp_location              : on_chip | off_chip;  
  
      auxiliary_output            : port_name;  
      auxiliary_output_enable     : port_name;  
      auxiliary_input             : port_name;  
      auxiliary_input_enable      : port_name;  
    }  
  }  
}
```

Description

This section describes the properties of a port *port_name* on the tcd_bscan module that will be directly connected to a top level port. The pad cell is inside the tcd_bscan segment and this wrapper describes the properties of the pad cell.

Arguments

- differential_inverse_of : *port_name*

This entry is used for the associated port of a differential pair. The *port_name* refers to the ExternalPort(*port_name*) wrapper of the representative port. This entry is exclusive with all other entries. The settings for the differential pair need to be done in the wrapper of the representative port.

- differential_type : voltage | current

Specifies the nature of the differential pair. If this is set to voltage it means that the signals are similar to the other logic signals. In case of current the differential pair is directly accessible from the tester and the tester needs to determine how to deal with this differential port. If this entry is specified there needs to be a ExternalPort() wrapper referring to this wrapper with the differential_inverse_of property to this port.

- **buffer_type** : three_state | two_state | low_only | high_only

This specifies the values the pad driver can drive and if the driver can be disabled. See the table for an overview of the combinations.

Table A-3. buffer types and their available state

buffer_type	can drive a 0	can drive a 1	can be disabled to Z
three_state	yes	yes	yes
two_state	yes	yes	no
low_only	yes	no	yes
high_only	no	yes	yes

A three_state port can drive 0 and 1 values and is enabled and disabled by an extra enable cell. A two_state port can drive a 0 and a 1, but cannot be disabled. Please note that you will not be able to implement a fully IEEE1149.1 HIGHZ instruction, if you use such a driver in your design, because this buffer type lacks the needed ability to disable the pad driver. The types low_only and high_only are asymmetric drivers like open emitter and open collector driver. The enable signal of a driver can be overridden by the force disable signal that switches off all driver except the two_state ones. This entry is only valid for output or inout ports.

- **highz_during_force_disable** : on | off | auto

Specifies if the pad driver can be disabled with the force disable signal. This entry is only valid for output or inout ports. In case of auto it is assumed that driver of the buffer_type three_state, low_only and high_only can be disabled.

- **pull_resistor** : high | low | none

Specifies if a pull resistor is present in the pad.

Table A-4. Valid combinations of the pull_resistor and buffer_type

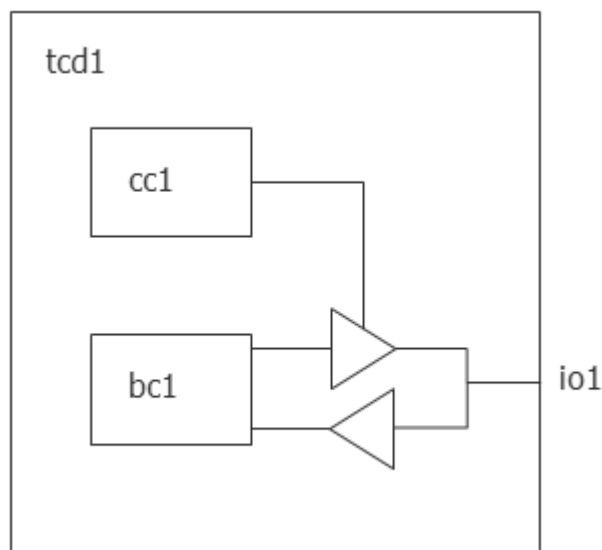
pull_resistor	buffer_type	bsdl disable result
none	three_state	Z
none	two_state	-
none	low_only	weak1
none	high_only	weak0
high	low_only	pull1
low	high_only	pull0

The table above shows which combinations are valid. This entry is only valid for output and inout ports.

- **control_cell** : *cell_id*
This refers to a Cell(*cell_id*) wrapper that contains the description of the control cell that enables and disables the pad. This entry is only valid for output and inout ports.
- **ac_hp_time** : *time*
This describes the AC high pass behavior of the pad according to the IEEE 1149.6 standard.
- **ac_lp_time** : *time*
This describes the AC low pass behavior of the pad according to the IEEE 1149.6 standard.
- **ac_hp_location** : *on_chip* | *off_chip*
This described if the AC high pass is on the chip or needs to be added on the outside.
- **auxiliary_output** : *port_name*
This specifies a port that can be multiplexed with the data from the core or the data bscan cell. The output of the multiplexer is connected to the pad of the external port. The external port needs to be an output or inout port. You need to specify also the **auxiliary_output_enable** entry.
- **auxiliary_output_enable** : *port_name*
This is the multiplexer enable for the **auxiliary_output** above. You need to specify both entries together.
- **auxiliary_input** : *port_name*
This specifies a port that gets the value from the pad of the external port, if enabled with the **auxiliary_input_enable**. The external port needs to be an input or inout port. You need to specify also the **auxiliary_input_enable** entry.
- **auxiliary_input_enable** : *port_name*
This is the enable for the **auxiliary_input** above. You need to specify both entries together.

Examples

Figure A-1. Bidirectional Pad with Data and Enable Cell



The core description of the bidirectional pad with boundary scan and enable cell :

```

Core(tcd1) {
  BoundaryScan {
    Interface { // not shown in the figure
      force_disable      : force_disable1;
      select_jtag_input  : select_jtag_input1;
      select_jtag_output : select_jtag_output1;
      bscan_clock        : bscan_clock1;
      capture_en         : capture_enable1;
      shift_en           : shift_enable1;
      update_en          : update_enable1;
      scan_in            : scan_input1;
      scan_out           : scan_output1;
      scan_out_launch_edge : negedge;
    }
    ExternalPort(io1) {
      buffer_type      : three_state;
      control_cell     : cc1;
    }
    Cell(cc1) {
      function          : control;
      bsd1_cell_type    : BC_2;
      control_enable_value : 1;
      safe_value        : 0;
    }
    Cell(bc1) {
      function          : bidir;
      bsd1_cell_type    : BC_7;
      external_port     : io1;
      safe_value        : 0;
    }
  }
}

```


Cell

This section describes a boundary scan cell.

Usage

```
Core(module_name) {
  BoundaryScan {
    Cell(id) {
      function                : output | input | bidir |
                              control | control_reset | internal |
                              observe_only | clock | ac_select;

      bsd1_cell_type          : cell_type_id;
      external_port           : port_name;
      control_enable_value   : 0 | 1;
      ac_select_cell         : cell_id;
      ac_select_port          : port_name;
      safe_value              : 0 | 1 | x;
      ac_type                 : on | off;
    }
  }
}
```

Description

This section describes a boundary scan cell.

Arguments

- function : output | input | bidir | control | control_reset | internal | observe_only | clock | ac_select

The following table shows the function of each boundary scan cell.

Table A-5. Cell functions

function	description	BSDL function
output	A data cell that drives an output external port. The cell may also be able to observe the value from the core.	OUTPUT2 or OUTPUT3
input	A data cell that observes an input external port. The cell may also be able to drive the signal to the core.	INPUT
bidir	A data cell for an inout external port.	BIDIR
control	A control cell that enables and disables a group of output and/or inout external ports.	CONTROL
control_reset	Same as control, but forced to the disabled state during the Test-Logic_reset TAP state.	CONTROLR

Table A-5. Cell functions

function	description	BSDL function
internal	Cell not associated with a external port.	INTERNAL
observe_only	A cell that captures values from input, output or inout external ports.	OBSERVE_ONLY
clock	A cell that observes the state of a clock external port.	CLOCK
ac_select	A cell that enables the AC functionality on a group of output and/or inout external ports.	INTERNAL

- **bsdl_cell_type** : *cell_type_id*
This is the entry that will be used to describe the cell in the BSDL file. This can either be a build in type like the BC_# or AC_# or a custom cell type described in a package file.
- **external_port** : *port_name*
This refers to an ExternalPort(*port_name*) wrapper. This cell is a data boundary scan cell of the external port.
-
- **control_enable_value** : 0 | 1
This specifies the enable value for cells with the function control, control_reset and ac_select. For those cells the entry is mandatory.
- **ac_select_cell** : *cell_id*
This refers to another Cell(*cell_id*) wrapper. The other cell is an AC select cell and will enable and disable the AC functionality of this cell.
- **safe_value** : 0 | 1 | x
This specified the save value of the cell. The safe value will be used in the BSDL file.
- **ac_type** : on | off
Specifiying this to on implies that this is an AC data boundary scan cell. It needs to have the function input, output, bidir, observe_only or clock.

Third-Party Information

For information about third-party software included with this release of Tessent products, refer to the [*Third-Party Software for Tessent Products*](#).



End-User License Agreement

The latest version of the End-User License Agreement is available on-line at:
www.mentor.com/eula

IMPORTANT INFORMATION

USE OF ALL SOFTWARE IS SUBJECT TO LICENSE RESTRICTIONS. CAREFULLY READ THIS LICENSE AGREEMENT BEFORE USING THE PRODUCTS. USE OF SOFTWARE INDICATES CUSTOMER'S COMPLETE AND UNCONDITIONAL ACCEPTANCE OF THE TERMS AND CONDITIONS SET FORTH IN THIS AGREEMENT. ANY ADDITIONAL OR DIFFERENT PURCHASE ORDER TERMS AND CONDITIONS SHALL NOT APPLY.

END-USER LICENSE AGREEMENT ("Agreement")

This is a legal agreement concerning the use of Software (as defined in Section 2) and hardware (collectively "Products") between the company acquiring the Products ("Customer"), and the Mentor Graphics entity that issued the corresponding quotation or, if no quotation was issued, the applicable local Mentor Graphics entity ("Mentor Graphics"). Except for license agreements related to the subject matter of this license agreement which are physically signed by Customer and an authorized representative of Mentor Graphics, this Agreement and the applicable quotation contain the parties' entire understanding relating to the subject matter and supersede all prior or contemporaneous agreements. If Customer does not agree to these terms and conditions, promptly return or, in the case of Software received electronically, certify destruction of Software and all accompanying items within five days after receipt of Software and receive a full refund of any license fee paid.

1. ORDERS, FEES AND PAYMENT.

- 1.1. To the extent Customer (or if agreed by Mentor Graphics, Customer's appointed third party buying agent) places and Mentor Graphics accepts purchase orders pursuant to this Agreement (each an "Order"), each Order will constitute a contract between Customer and Mentor Graphics, which shall be governed solely and exclusively by the terms and conditions of this Agreement, any applicable addenda and the applicable quotation, whether or not those documents are referenced on the Order. Any additional or conflicting terms and conditions appearing on an Order or presented in any electronic portal or automated order management system, whether or not required to be electronically accepted, will not be effective unless agreed in writing and physically signed by an authorized representative of Customer and Mentor Graphics.
- 1.2. Amounts invoiced will be paid, in the currency specified on the applicable invoice, within 30 days from the date of such invoice. Any past due invoices will be subject to the imposition of interest charges in the amount of one and one-half percent per month or the applicable legal rate currently in effect, whichever is lower. Prices do not include freight, insurance, customs duties, taxes or other similar charges, which Mentor Graphics will state separately in the applicable invoice. Unless timely provided with a valid certificate of exemption or other evidence that items are not taxable, Mentor Graphics will invoice Customer for all applicable taxes including, but not limited to, VAT, GST, sales tax, consumption tax and service tax. Customer will make all payments free and clear of, and without reduction for, any withholding or other taxes; any such taxes imposed on payments by Customer hereunder will be Customer's sole responsibility. If Customer appoints a third party to place purchase orders and/or make payments on Customer's behalf, Customer shall be liable for payment under Orders placed by such third party in the event of default.
- 1.3. All Products are delivered FCA factory (Incoterms 2010), freight prepaid and invoiced to Customer, except Software delivered electronically, which shall be deemed delivered when made available to Customer for download. Mentor Graphics retains a security interest in all Products delivered under this Agreement, to secure payment of the purchase price of such Products, and Customer agrees to sign any documents that Mentor Graphics determines to be necessary or convenient for use in filing or perfecting such security interest. Mentor Graphics' delivery of Software by electronic means is subject to Customer's provision of both a primary and an alternate e-mail address.

2. **GRANT OF LICENSE.** The software installed, downloaded, or otherwise acquired by Customer under this Agreement, including any updates, modifications, revisions, copies, documentation, setup files and design data ("Software") are copyrighted, trade secret and confidential information of Mentor Graphics or its licensors, who maintain exclusive title to all Software and retain all rights not expressly granted by this Agreement. Except for Software that is embeddable ("Embedded Software"), which is licensed pursuant to separate embedded software terms or an embedded software supplement, Mentor Graphics grants to Customer, subject to payment of applicable license fees, a nontransferable, nonexclusive license to use Software solely: (a) in machine-readable, object-code form (except as provided in Subsection 4.2); (b) for Customer's internal business purposes; (c) for the term of the license; and (d) on the computer hardware and at the site authorized by Mentor Graphics. A site is restricted to a one-half mile (800 meter) radius. Customer may have Software temporarily used by an employee for telecommuting purposes from locations other than a Customer office, such as the employee's residence, an airport or hotel, provided that such employee's primary place of employment is the site where the Software is authorized for use. Mentor Graphics' standard policies and programs, which vary depending on Software, license fees paid or services purchased, apply to the following: (a) relocation of Software; (b) use of Software, which may be limited, for example, to execution of a single session by a single user on the authorized hardware or for a restricted period of time (such limitations may be technically implemented through the use of authorization codes or similar devices); and (c) support services provided, including eligibility to receive telephone support, updates, modifications, and revisions. For the avoidance of doubt, if Customer provides any feedback or requests any change or enhancement to Products, whether in the course of receiving support or consulting services, evaluating Products, performing beta testing or otherwise, any inventions, product improvements, modifications or developments made by Mentor Graphics (at Mentor Graphics' sole discretion) will be the exclusive property of Mentor Graphics.

3. BETA CODE.

- 3.1. Portions of all of certain Software may contain code for experimental testing and evaluation (which may be either alpha or beta, collectively "Beta Code"), which may not be used without Mentor Graphics' explicit authorization. Upon Mentor Graphics' authorization, Mentor Graphics grants to Customer a temporary, nontransferable, nonexclusive license for experimental use to test and evaluate the Beta Code without charge for a limited period of time specified by Mentor Graphics. Mentor Graphics may choose, at its sole discretion, not to release Beta Code commercially in any form.
- 3.2. If Mentor Graphics authorizes Customer to use the Beta Code, Customer agrees to evaluate and test the Beta Code under normal conditions as directed by Mentor Graphics. Customer will contact Mentor Graphics periodically during Customer's use of the Beta Code to discuss any malfunctions or suggested improvements. Upon completion of Customer's evaluation and testing, Customer will send to Mentor Graphics a written evaluation of the Beta Code, including its strengths, weaknesses and recommended improvements.
- 3.3. Customer agrees to maintain Beta Code in confidence and shall restrict access to the Beta Code, including the methods and concepts utilized therein, solely to those employees and Customer location(s) authorized by Mentor Graphics to perform beta testing. Customer agrees that any written evaluations and all inventions, product improvements, modifications or developments that Mentor Graphics conceived or made during or subsequent to this Agreement, including those based partly or wholly on Customer's feedback, will be the exclusive property of Mentor Graphics. Mentor Graphics will have exclusive rights, title and interest in all such property. The provisions of this Subsection 3.3 shall survive termination of this Agreement.

4. RESTRICTIONS ON USE.

- 4.1. Customer may copy Software only as reasonably necessary to support the authorized use. Each copy must include all notices and legends embedded in Software and affixed to its medium and container as received from Mentor Graphics. All copies shall remain the property of Mentor Graphics or its licensors. Except for Embedded Software that has been embedded in executable code form in Customer's product(s), Customer shall maintain a record of the number and primary location of all copies of Software, including copies merged with other software, and shall make those records available to Mentor Graphics upon request. Customer shall not make Products available in any form to any person other than Customer's employees and on-site contractors, excluding Mentor Graphics competitors, whose job performance requires access and who are under obligations of confidentiality. Customer shall take appropriate action to protect the confidentiality of Products and ensure that any person permitted access does not disclose or use Products except as permitted by this Agreement. Customer shall give Mentor Graphics written notice of any unauthorized disclosure or use of the Products as soon as Customer becomes aware of such unauthorized disclosure or use. Customer acknowledges that Software provided hereunder may contain source code which is proprietary and its confidentiality is of the highest importance and value to Mentor Graphics. Customer acknowledges that Mentor Graphics may be seriously harmed if such source code is disclosed in violation of this Agreement. Except as otherwise permitted for purposes of interoperability as specified by applicable and mandatory local law, Customer shall not reverse-assemble, disassemble, reverse-compile, or reverse-engineer any Product, or in any way derive any source code from Software that is not provided to Customer in source code form. Log files, data files, rule files and script files generated by or for the Software (collectively "Files"), including without limitation files containing Standard Verification Rule Format ("SVRF") and Tcl Verification Format ("TVF") which are Mentor Graphics' trade secret and proprietary syntaxes for expressing process rules, constitute or include confidential information of Mentor Graphics. Customer may share Files with third parties, excluding Mentor Graphics competitors, provided that the confidentiality of such Files is protected by written agreement at least as well as Customer protects other information of a similar nature or importance, but in any case with at least reasonable care. Customer may use Files containing SVRF or TVF only with Mentor Graphics products. Under no circumstances shall Customer use Products or Files or allow their use for the purpose of developing, enhancing or marketing any product that is in any way competitive with Products, or disclose to any third party the results of, or information pertaining to, any benchmark.
 - 4.2. If any Software or portions thereof are provided in source code form, Customer will use the source code only to correct software errors and enhance or modify the Software for the authorized use, or as permitted for Embedded Software under separate embedded software terms or an embedded software supplement. Customer shall not disclose or permit disclosure of source code, in whole or in part, including any of its methods or concepts, to anyone except Customer's employees or on-site contractors, excluding Mentor Graphics competitors, with a need to know. Customer shall not copy or compile source code in any manner except to support this authorized use.
 - 4.3. Customer agrees that it will not subject any Product to any open source software ("OSS") license that conflicts with this Agreement or that does not otherwise apply to such Product.
 - 4.4. Customer may not assign this Agreement or the rights and duties under it, or relocate, sublicense, or otherwise transfer the Products, whether by operation of law or otherwise ("Attempted Transfer"), without Mentor Graphics' prior written consent and payment of Mentor Graphics' then-current applicable relocation and/or transfer fees. Any Attempted Transfer without Mentor Graphics' prior written consent shall be a material breach of this Agreement and may, at Mentor Graphics' option, result in the immediate termination of the Agreement and/or the licenses granted under this Agreement. The terms of this Agreement, including without limitation the licensing and assignment provisions, shall be binding upon Customer's permitted successors in interest and assigns.
 - 4.5. The provisions of this Section 4 shall survive the termination of this Agreement.
5. **SUPPORT SERVICES.** To the extent Customer purchases support services, Mentor Graphics will provide Customer with updates and technical support for the Products, at the Customer site(s) for which support is purchased, in accordance with Mentor Graphics' then current End-User Support Terms located at <http://supportnet.mentor.com/supportterms>.
6. **OPEN SOURCE SOFTWARE.** Products may contain OSS or code distributed under a proprietary third party license agreement, to which additional rights or obligations ("Third Party Terms") may apply. Please see the applicable Product documentation (including license files, header files, read-me files or source code) for details. In the event of conflict between the terms of this Agreement

(including any addenda) and the Third Party Terms, the Third Party Terms will control solely with respect to the OSS or third party code. The provisions of this Section 6 shall survive the termination of this Agreement.

7. LIMITED WARRANTY.

- 7.1. Mentor Graphics warrants that during the warranty period its standard, generally supported Products, when properly installed, will substantially conform to the functional specifications set forth in the applicable user manual. Mentor Graphics does not warrant that Products will meet Customer's requirements or that operation of Products will be uninterrupted or error free. The warranty period is 90 days starting on the 15th day after delivery or upon installation, whichever first occurs. Customer must notify Mentor Graphics in writing of any nonconformity within the warranty period. For the avoidance of doubt, this warranty applies only to the initial shipment of Software under an Order and does not renew or reset, for example, with the delivery of (a) Software updates or (b) authorization codes or alternate Software under a transaction involving Software re-mix. This warranty shall not be valid if Products have been subject to misuse, unauthorized modification, improper installation or Customer is not in compliance with this Agreement. MENTOR GRAPHICS' ENTIRE LIABILITY AND CUSTOMER'S EXCLUSIVE REMEDY SHALL BE, AT MENTOR GRAPHICS' OPTION, EITHER (A) REFUND OF THE PRICE PAID UPON RETURN OF THE PRODUCTS TO MENTOR GRAPHICS OR (B) MODIFICATION OR REPLACEMENT OF THE PRODUCTS THAT DO NOT MEET THIS LIMITED WARRANTY. MENTOR GRAPHICS MAKES NO WARRANTIES WITH RESPECT TO: (A) SERVICES; (B) PRODUCTS PROVIDED AT NO CHARGE; OR (C) BETA CODE; ALL OF WHICH ARE PROVIDED "AS IS."
- 7.2. THE WARRANTIES SET FORTH IN THIS SECTION 7 ARE EXCLUSIVE. NEITHER MENTOR GRAPHICS NOR ITS LICENSORS MAKE ANY OTHER WARRANTIES EXPRESS, IMPLIED OR STATUTORY, WITH RESPECT TO PRODUCTS PROVIDED UNDER THIS AGREEMENT. MENTOR GRAPHICS AND ITS LICENSORS SPECIFICALLY DISCLAIM ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY.

8. **LIMITATION OF LIABILITY.** TO THE EXTENT PERMITTED UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. IN NO EVENT SHALL MENTOR GRAPHICS' OR ITS LICENSORS' LIABILITY UNDER THIS AGREEMENT EXCEED THE AMOUNT RECEIVED FROM CUSTOMER FOR THE HARDWARE, SOFTWARE LICENSE OR SERVICE GIVING RISE TO THE CLAIM. IN THE CASE WHERE NO AMOUNT WAS PAID, MENTOR GRAPHICS AND ITS LICENSORS SHALL HAVE NO LIABILITY FOR ANY DAMAGES WHATSOEVER. THE PROVISIONS OF THIS SECTION 8 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

9. THIRD PARTY CLAIMS.

- 9.1. Customer acknowledges that Mentor Graphics has no control over the testing of Customer's products, or the specific applications and use of Products. Mentor Graphics and its licensors shall not be liable for any claim or demand made against Customer by any third party, except to the extent such claim is covered under Section 10.
- 9.2. In the event that a third party makes a claim against Mentor Graphics arising out of the use of Customer's products, Mentor Graphics will give Customer prompt notice of such claim. At Customer's option and expense, Customer may take sole control of the defense and any settlement of such claim. Customer WILL reimburse and hold harmless Mentor Graphics for any LIABILITY, damages, settlement amounts, costs and expenses, including reasonable attorney's fees, incurred by or awarded against Mentor Graphics or its licensors in connection with such claims.
- 9.3. The provisions of this Section 9 shall survive any expiration or termination of this Agreement.

10. INFRINGEMENT.

- 10.1. Mentor Graphics will defend or settle, at its option and expense, any action brought against Customer in the United States, Canada, Japan, or member state of the European Union which alleges that any standard, generally supported Product acquired by Customer hereunder infringes a patent or copyright or misappropriates a trade secret in such jurisdiction. Mentor Graphics will pay costs and damages finally awarded against Customer that are attributable to such action. Customer understands and agrees that as conditions to Mentor Graphics' obligations under this section Customer must: (a) notify Mentor Graphics promptly in writing of the action; (b) provide Mentor Graphics all reasonable information and assistance to settle or defend the action; and (c) grant Mentor Graphics sole authority and control of the defense or settlement of the action.
- 10.2. If a claim is made under Subsection 10.1 Mentor Graphics may, at its option and expense: (a) replace or modify the Product so that it becomes noninfringing; (b) procure for Customer the right to continue using the Product; or (c) require the return of the Product and refund to Customer any purchase price or license fee paid, less a reasonable allowance for use.
- 10.3. Mentor Graphics has no liability to Customer if the action is based upon: (a) the combination of Software or hardware with any product not furnished by Mentor Graphics; (b) the modification of the Product other than by Mentor Graphics; (c) the use of other than a current unaltered release of Software; (d) the use of the Product as part of an infringing process; (e) a product that Customer makes, uses, or sells; (f) any Beta Code or Product provided at no charge; (g) any software provided by Mentor Graphics' licensors who do not provide such indemnification to Mentor Graphics' customers; (h) OSS, except to the extent that the infringement is directly caused by Mentor Graphics' modifications to such OSS; or (i) infringement by Customer that is deemed willful. In the case of (i), Customer shall reimburse Mentor Graphics for its reasonable attorney fees and other costs related to the action.
- 10.4. THIS SECTION 10 IS SUBJECT TO SECTION 8 ABOVE AND STATES THE ENTIRE LIABILITY OF MENTOR GRAPHICS AND ITS LICENSORS, AND CUSTOMER'S SOLE AND EXCLUSIVE REMEDY, FOR DEFENSE, SETTLEMENT AND DAMAGES, WITH RESPECT TO ANY ALLEGED PATENT OR COPYRIGHT INFRINGEMENT OR TRADE SECRET MISAPPROPRIATION BY ANY PRODUCT PROVIDED UNDER THIS AGREEMENT.

11. TERMINATION AND EFFECT OF TERMINATION.

- 11.1. If a Software license was provided for limited term use, such license will automatically terminate at the end of the authorized term. Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement immediately upon written notice if Customer: (a) exceeds the scope of the license or otherwise fails to comply with the licensing or confidentiality provisions of this Agreement, or (b) becomes insolvent, files a bankruptcy petition, institutes proceedings for liquidation or winding up or enters into an agreement to assign its assets for the benefit of creditors. For any other material breach of any provision of this Agreement, Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement upon 30 days written notice if Customer fails to cure the breach within the 30 day notice period. Termination of this Agreement or any license granted hereunder will not affect Customer's obligation to pay for Products shipped or licenses granted prior to the termination, which amounts shall be payable immediately upon the date of termination.
- 11.2. Upon termination of this Agreement, the rights and obligations of the parties shall cease except as expressly set forth in this Agreement. Upon termination of this Agreement and/or any license granted under this Agreement, Customer shall ensure that all use of the affected Products ceases, and shall return hardware and either return to Mentor Graphics or destroy Software in Customer's possession, including all copies and documentation, and certify in writing to Mentor Graphics within ten business days of the termination date that Customer no longer possesses any of the affected Products or copies of Software in any form.
12. **EXPORT.** The Products provided hereunder are subject to regulation by local laws and European Union ("E.U.") and United States ("U.S.") government agencies, which prohibit export, re-export or diversion of certain products, information about the products, and direct or indirect products thereof, to certain countries and certain persons. Customer agrees that it will not export or re-export Products in any manner without first obtaining all necessary approval from appropriate local, E.U. and U.S. government agencies. If Customer wishes to disclose any information to Mentor Graphics that is subject to any E.U., U.S. or other applicable export restrictions, including without limitation the U.S. International Traffic in Arms Regulations (ITAR) or special controls under the Export Administration Regulations (EAR), Customer will notify Mentor Graphics personnel, in advance of each instance of disclosure, that such information is subject to such export restrictions.
13. **U.S. GOVERNMENT LICENSE RIGHTS.** Software was developed entirely at private expense. The parties agree that all Software is commercial computer software within the meaning of the applicable acquisition regulations. Accordingly, pursuant to U.S. FAR 48 CFR 12.212 and DFAR 48 CFR 227.7202, use, duplication and disclosure of the Software by or for the U.S. government or a U.S. government subcontractor is subject solely to the terms and conditions set forth in this Agreement, which shall supersede any conflicting terms or conditions in any government order document, except for provisions which are contrary to applicable mandatory federal laws.
14. **THIRD PARTY BENEFICIARY.** Mentor Graphics Corporation, Mentor Graphics (Ireland) Limited, Microsoft Corporation and other licensors may be third party beneficiaries of this Agreement with the right to enforce the obligations set forth herein.
15. **REVIEW OF LICENSE USAGE.** Customer will monitor the access to and use of Software. With prior written notice and during Customer's normal business hours, Mentor Graphics may engage an internationally recognized accounting firm to review Customer's software monitoring system and records deemed relevant by the internationally recognized accounting firm to confirm Customer's compliance with the terms of this Agreement or U.S. or other local export laws. Such review may include FlexNet (or successor product) report log files that Customer shall capture and provide at Mentor Graphics' request. Customer shall make records available in electronic format and shall fully cooperate with data gathering to support the license review. Mentor Graphics shall bear the expense of any such review unless a material non-compliance is revealed. Mentor Graphics shall treat as confidential information all information gained as a result of any request or review and shall only use or disclose such information as required by law or to enforce its rights under this Agreement. The provisions of this Section 15 shall survive the termination of this Agreement.
16. **CONTROLLING LAW, JURISDICTION AND DISPUTE RESOLUTION.** The owners of certain Mentor Graphics intellectual property licensed under this Agreement are located in Ireland and the U.S. To promote consistency around the world, disputes shall be resolved as follows: excluding conflict of laws rules, this Agreement shall be governed by and construed under the laws of the State of Oregon, U.S., if Customer is located in North or South America, and the laws of Ireland if Customer is located outside of North or South America or Japan, and the laws of Japan if Customer is located in Japan. All disputes arising out of or in relation to this Agreement shall be submitted to the exclusive jurisdiction of the courts of Portland, Oregon when the laws of Oregon apply, or Dublin, Ireland when the laws of Ireland apply, or the Tokyo District Court when the laws of Japan apply. Notwithstanding the foregoing, all disputes in Asia (excluding Japan) arising out of or in relation to this Agreement shall be resolved by arbitration in Singapore before a single arbitrator to be appointed by the chairman of the Singapore International Arbitration Centre ("SIAC") to be conducted in the English language, in accordance with the Arbitration Rules of the SIAC in effect at the time of the dispute, which rules are deemed to be incorporated by reference in this section. Nothing in this section shall restrict Mentor Graphics' right to bring an action (including for example a motion for injunctive relief) against Customer in the jurisdiction where Customer's place of business is located. The United Nations Convention on Contracts for the International Sale of Goods does not apply to this Agreement.
17. **SEVERABILITY.** If any provision of this Agreement is held by a court of competent jurisdiction to be void, invalid, unenforceable or illegal, such provision shall be severed from this Agreement and the remaining provisions will remain in full force and effect.
18. **MISCELLANEOUS.** This Agreement contains the parties' entire understanding relating to its subject matter and supersedes all prior or contemporaneous agreements. Any translation of this Agreement is provided to comply with local legal requirements only. In the event of a dispute between the English and any non-English versions, the English version of this Agreement shall govern to the extent not prohibited by local law in the applicable jurisdiction. This Agreement may only be modified in writing, signed by an authorized representative of each party. Waiver of terms or excuse of breach must be in writing and shall not constitute subsequent consent, waiver or excuse.